

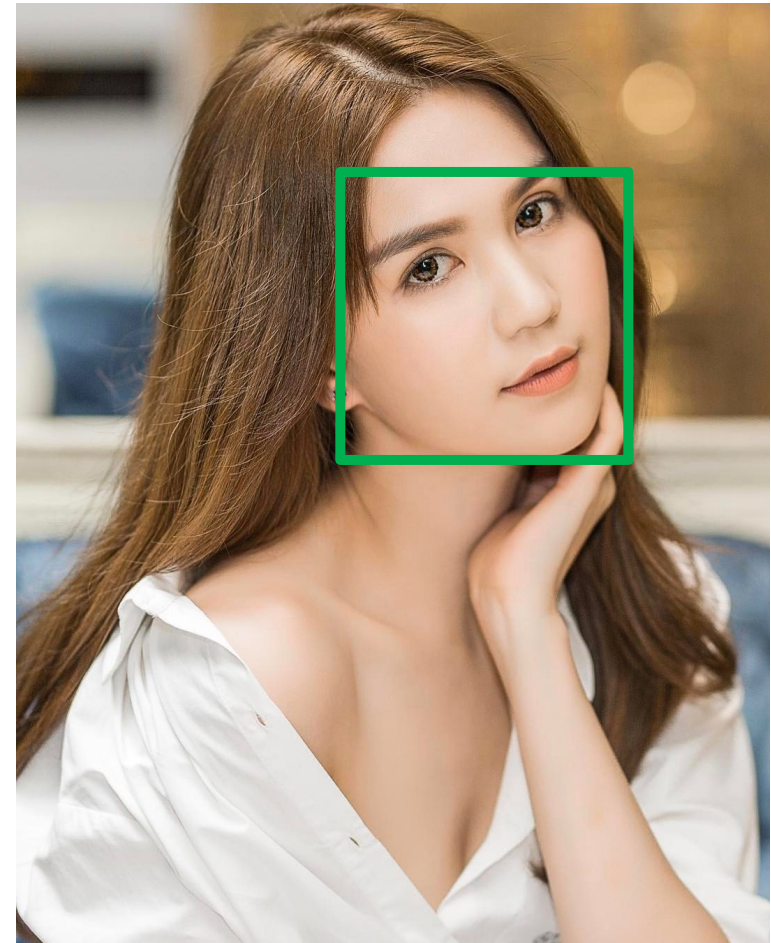
Object Detection

Topics in Computer Vision



Outline

- What is object detection?
- Sibling Window
- Selective Search
- Deep learning approach
- Conclusion



Detection Competitions

Pascal VOC

COCO

VOC20 classes



COCO: 200
classes

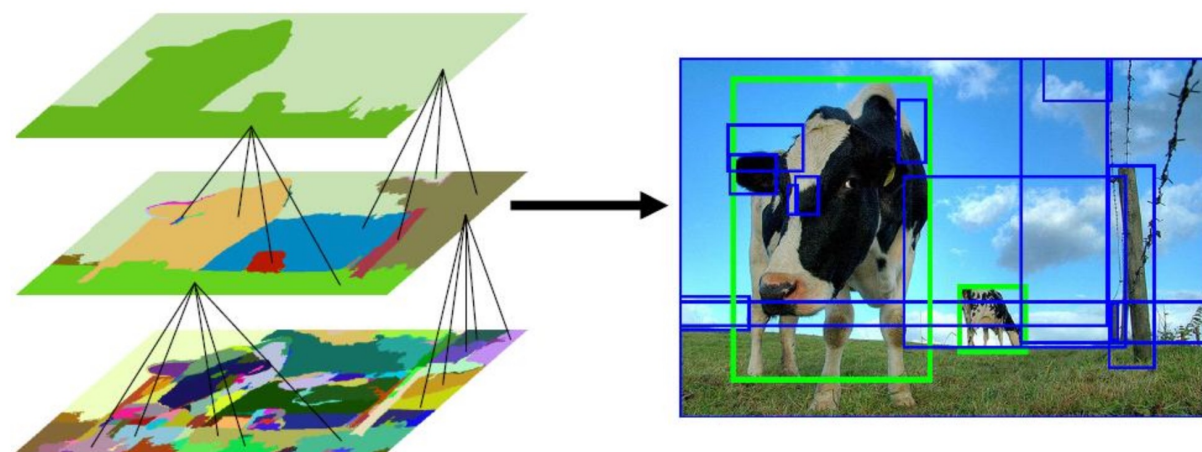


Before deep learning

- Sliding windows

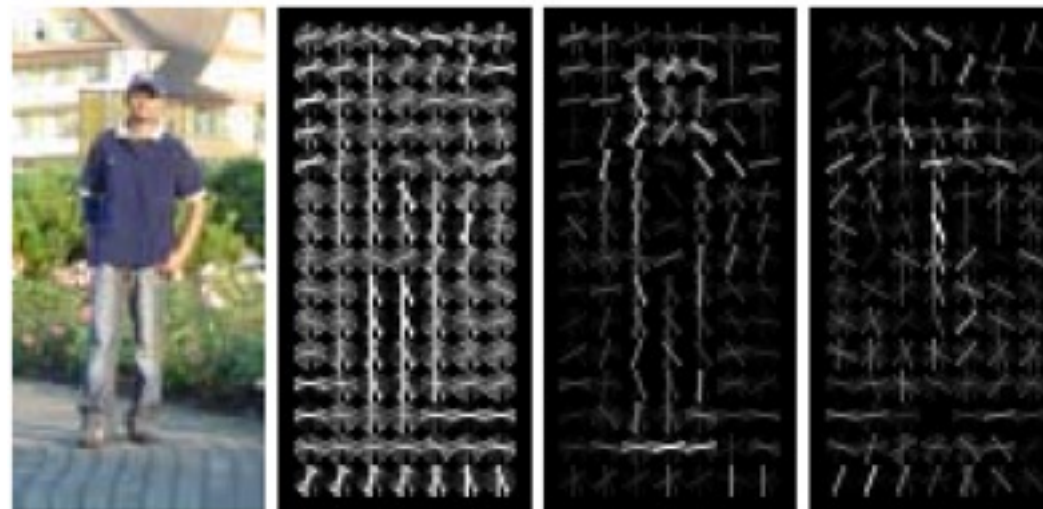


- Selective search

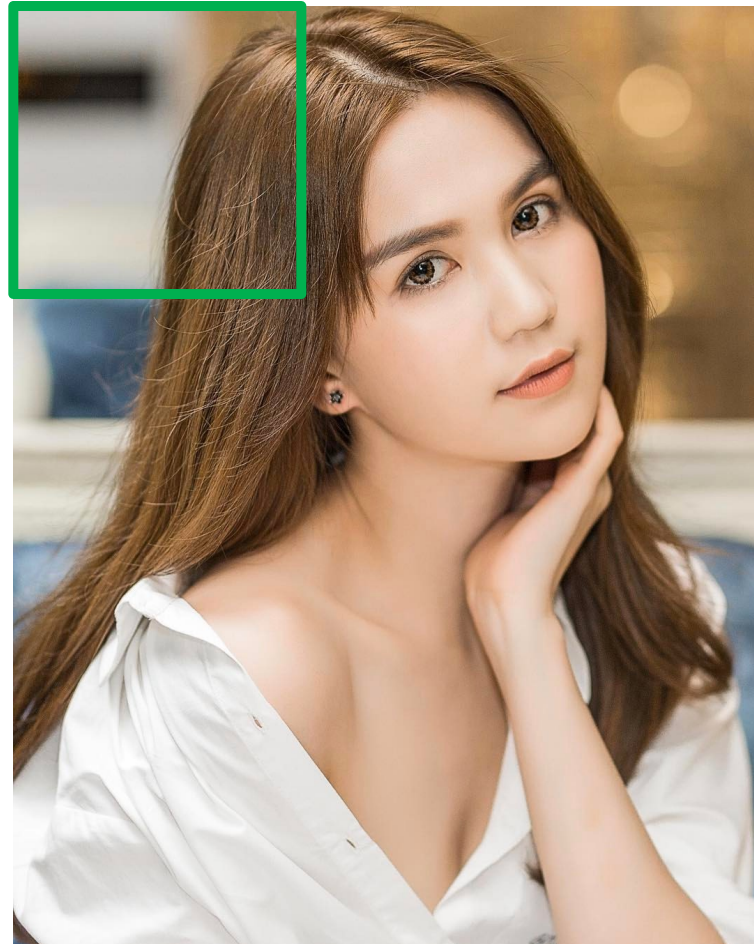


Before deep learning

- Sliding windows



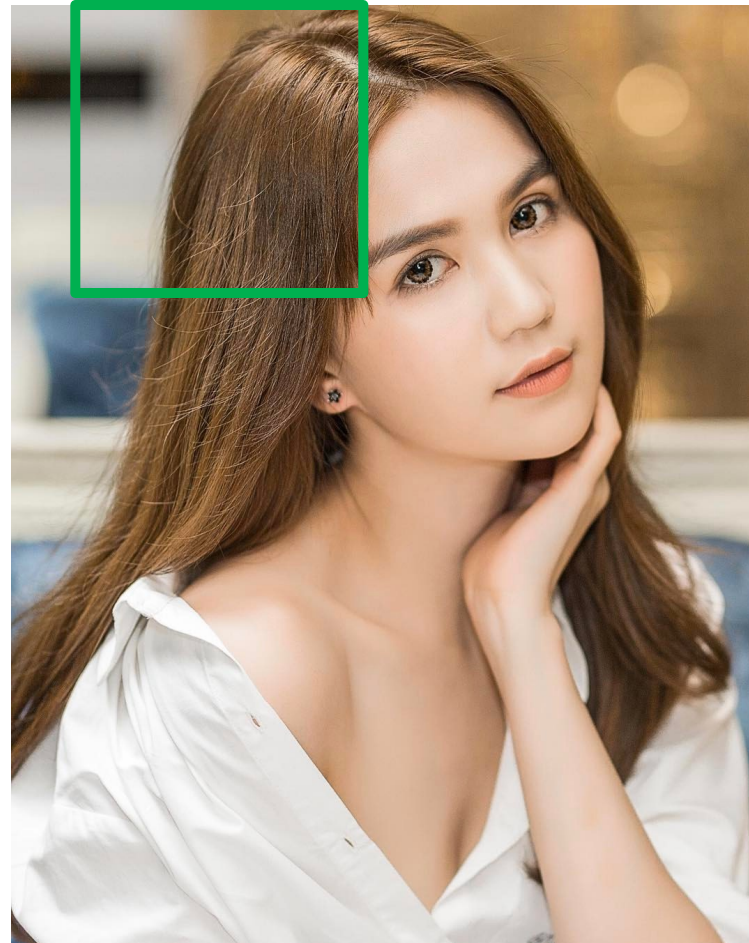
Sliding windows



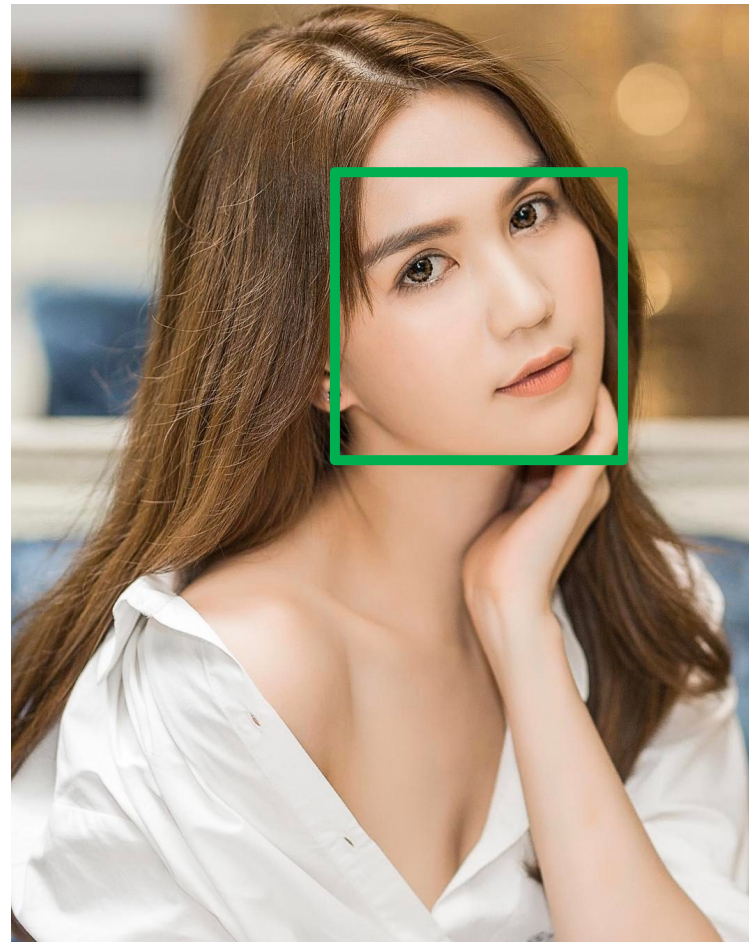
Sliding windows



Sliding windows



Sliding windows



Tips

Hard Negative Mining

Imbalance between positive and negative examples.

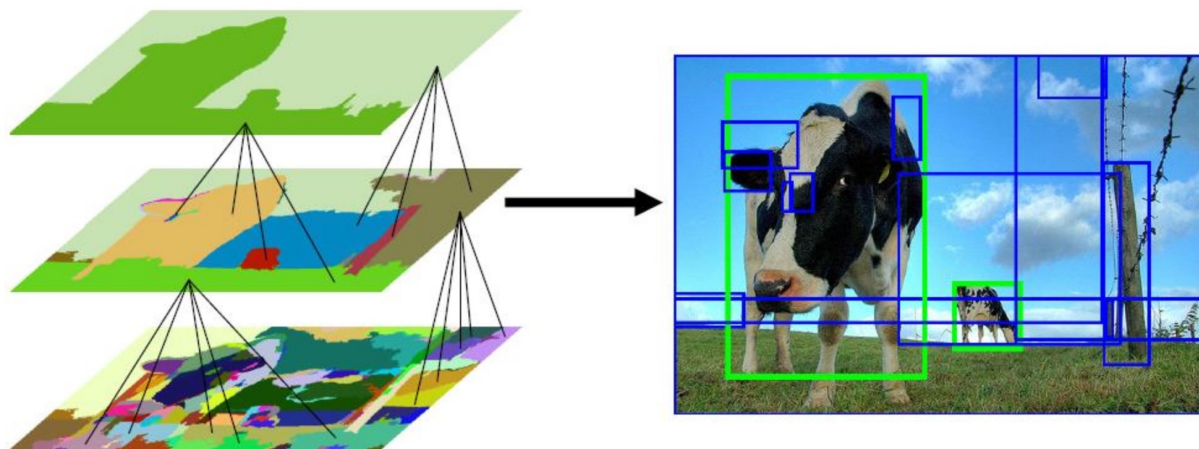
Use negative examples with higher confidence score.

Non Maximum Suppression

If output boxes overlap, only consider the most confident.

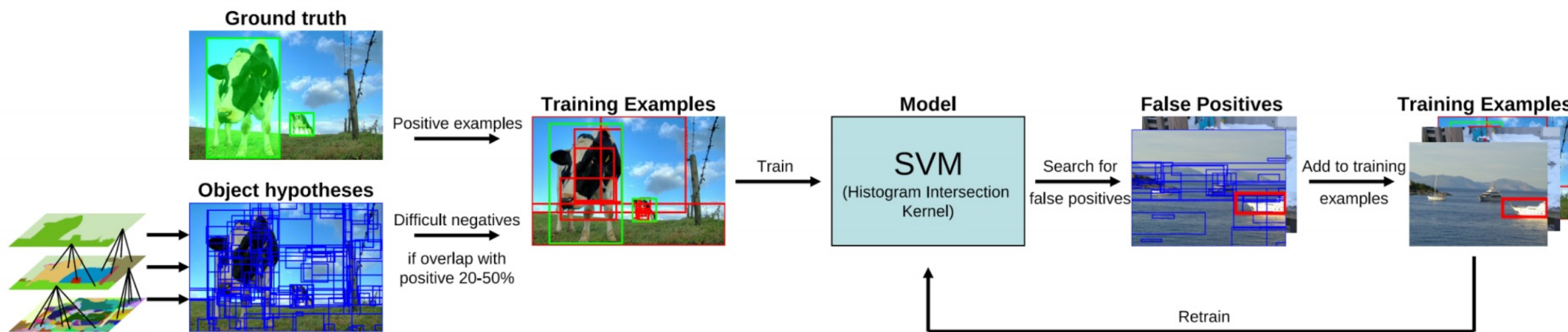
Before deep learning

- Selective search



Before deep learning

○ Selective Search



Selective Search



Selective Search

Goals:

1. Detect objects at any scale.
 - a. Hierarchical algorithms are good at this.
2. Consider multiple grouping criteria.
 - a. Detect differences in color, texture, brightness, etc.
3. Be fast.

Idea:

Use bottom-up grouping of image regions to generate a hierarchy of small to large regions.

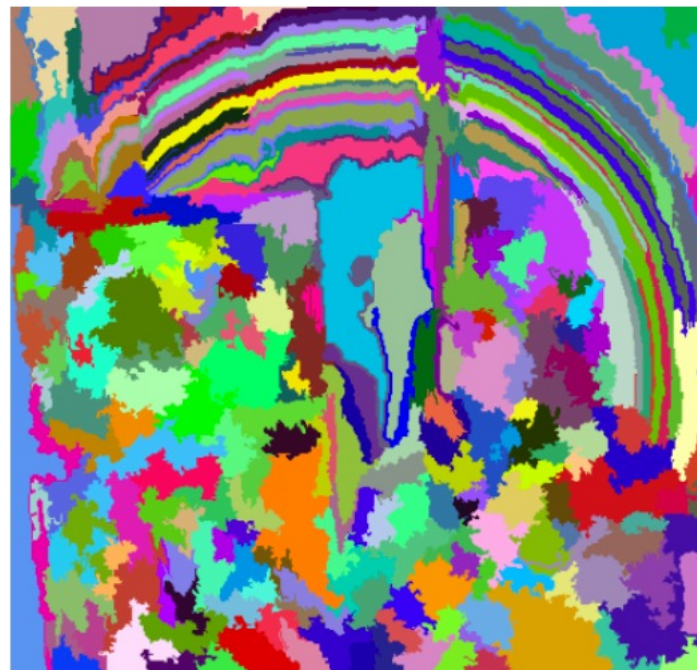
Selective Search

Step 1: Generate initial sub-segmentation

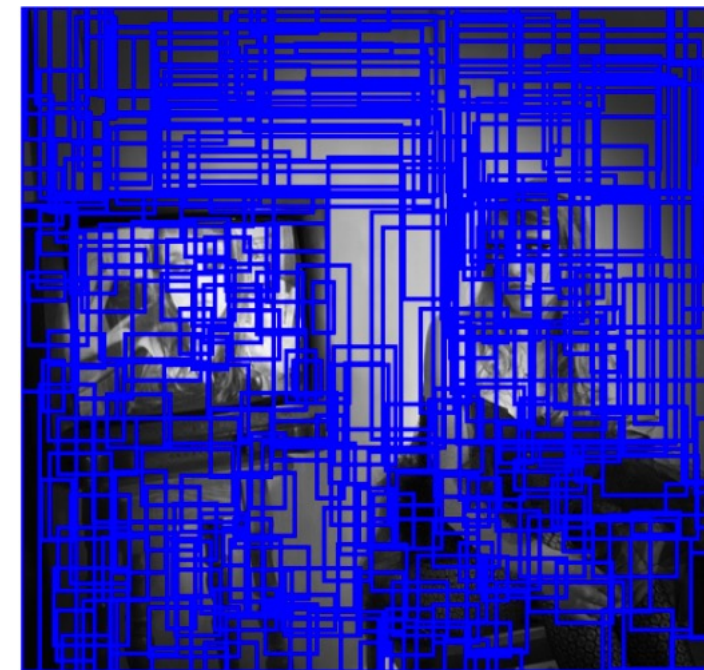
Goal: Generate many regions, each of which belongs to at most one object.



Input



Segmentation



Candidate objects

Selective Search

Step 2: Recursively combine similar regions into larger ones. This yields a hierarchy of successively larger regions

Greedy algorithm:

1. From set of regions, choose two that are most similar.
2. Combine them into a single, larger region.
3. Repeat until only one region remains.

This yields a hierarchy of successively larger regions, just like we want.

Selective Search

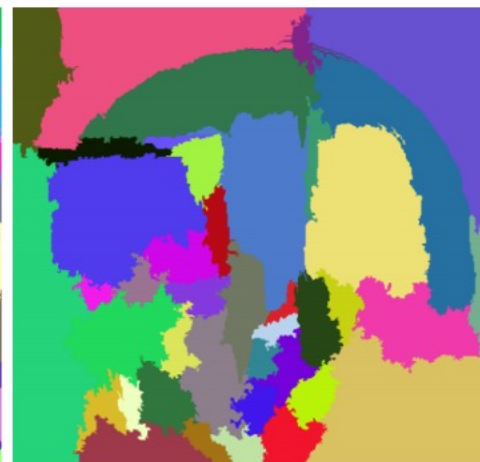
Step 2: Recursively combine similar regions into larger ones. This yields a hierarchy of successively larger regions



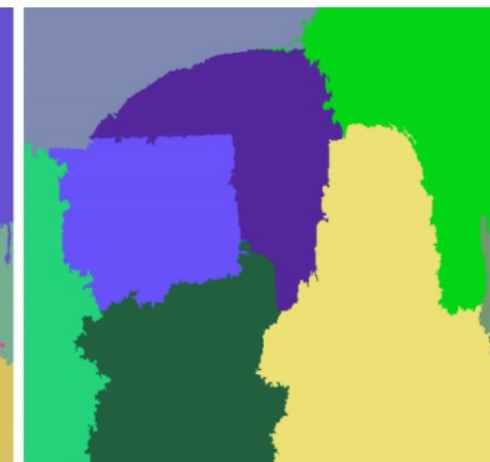
Input Image



Initial Segmentation



After some
iterations



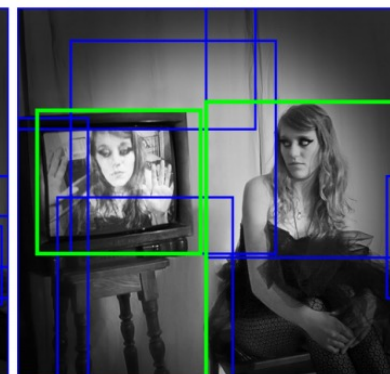
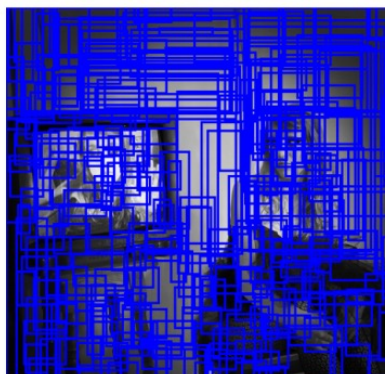
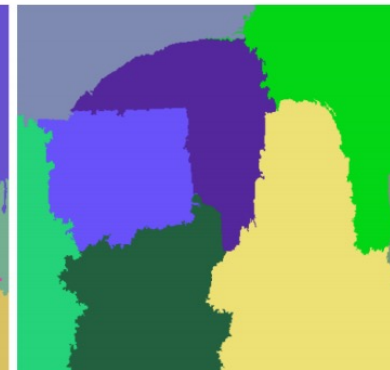
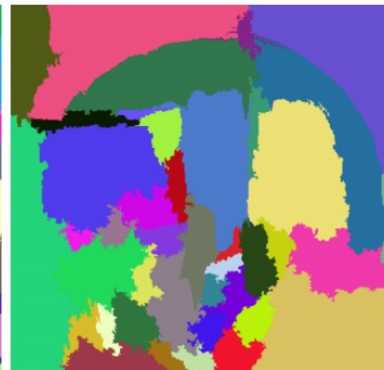
After more
iterations

Selective Search

Step 3: Use the generated regions to produce candidate object locations.



Input Image



Similarity

What do we mean by “similarity”?

Goals:

1. Use multiple grouping criteria.
2. Lead to a balanced hierarchy of small to large objects.
3. Be efficient to compute: should be able to quickly combine measurements in two regions.

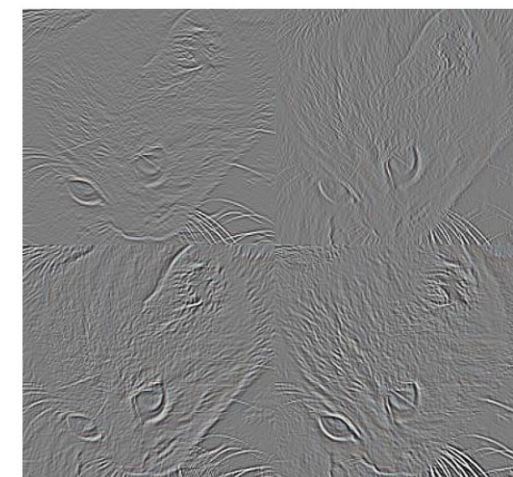
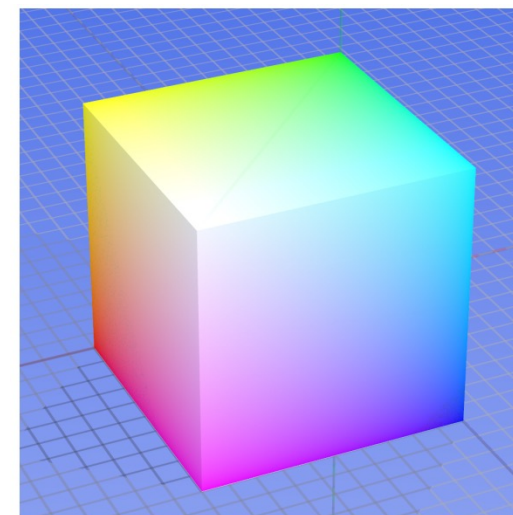


Similarity

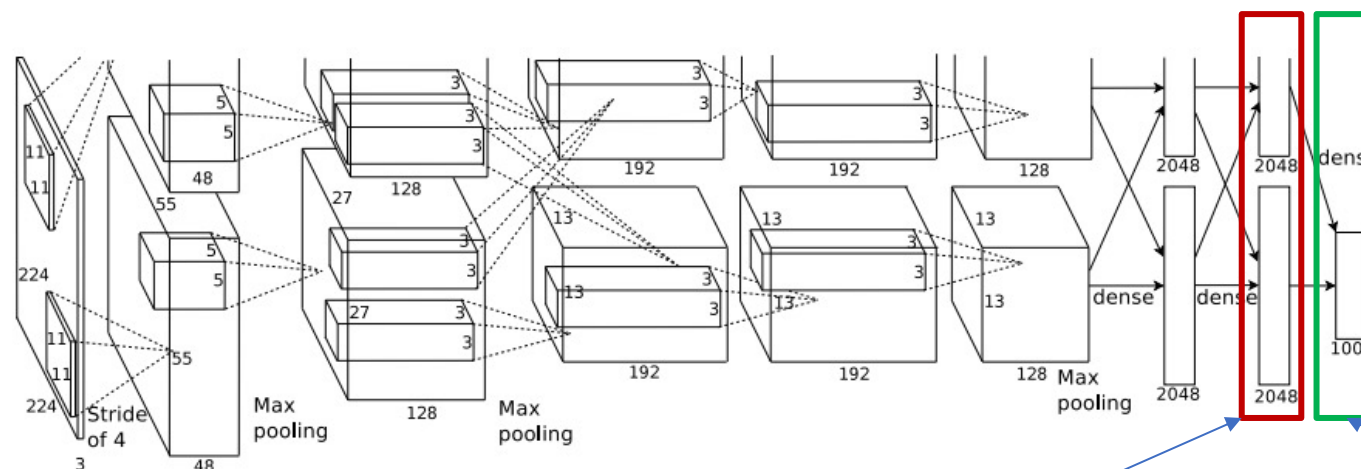
What do we mean by “similarity”?

Two-pronged approach:

1. Choose a color space that captures interesting thing. Different color spaces have different invariants, and different responses to changes in color.
2. Choose a similarity metric for that space that captures everything we’re interested: color, texture, size, and shape.



So far: Image Classification



Class Scores

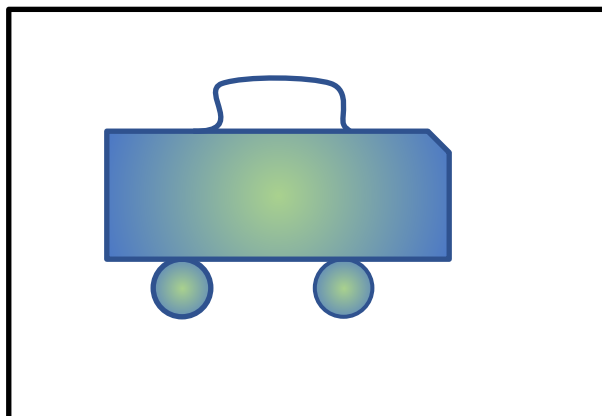
Cat: 0.9
Dog: 0.05
Car: 0.01 ...

Vector:
4096

Fully-Connected:
4096 to 1000

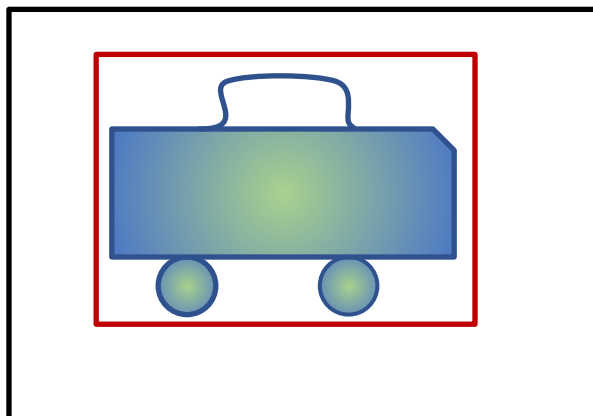
What is Object Detection?

Classification



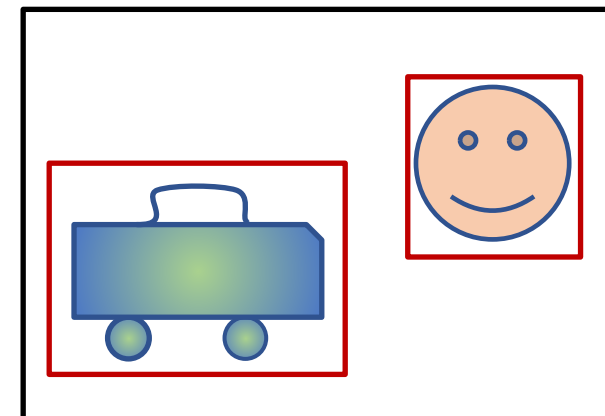
Car

Classification & Localization



Car

Object Detection

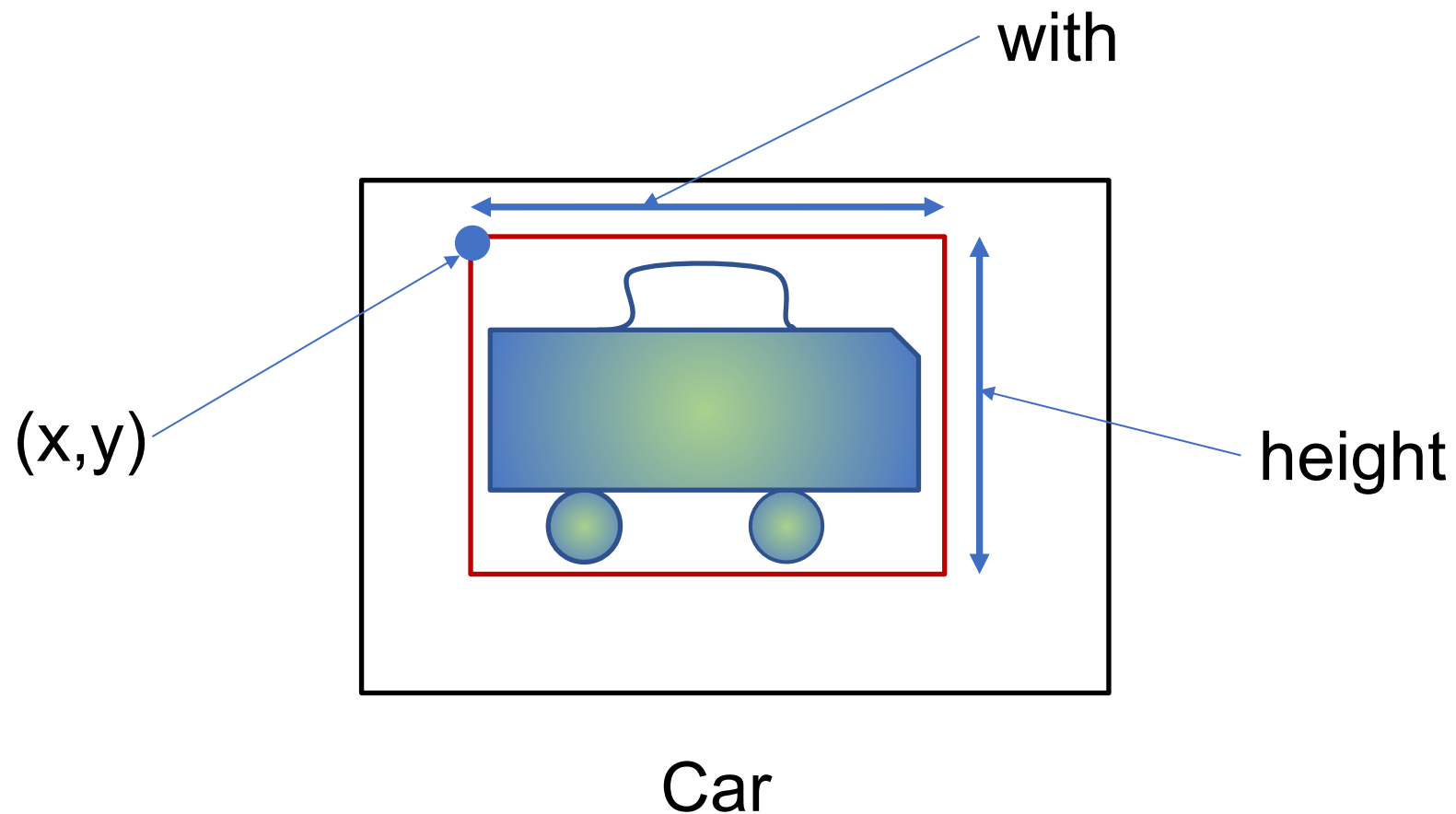


Car, Person

Single object

multiple object

Object Localization & Classification



Classification + Localization: ImageNet

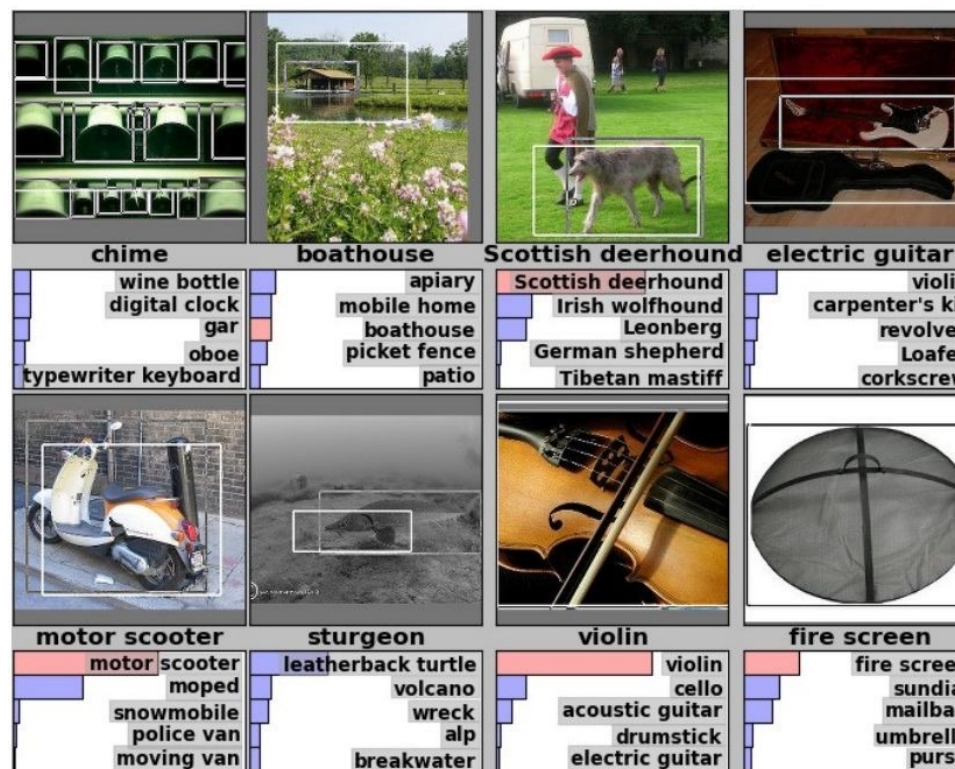
1000 classes (same as classification)

Each image has 1 class, at least one bounding box

~800 training images per class

Algorithm produces 5 (class, box) guesses

Example is correct if at least one one guess has correct class AND bounding box at least 0.5 intersection over union (IoU)



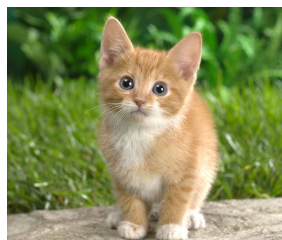
Classification + Localization: Task

Classification: C classes

Input: Image

Output: Class label

Evaluation metric: Accuracy



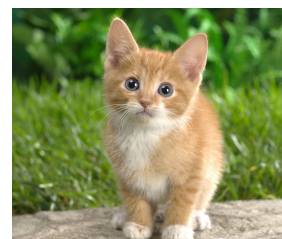
→ CAT

Localization:

Input: Image

Output: Box in the image (x, y, w, h)

Evaluation metric: Intersection over Union



→ (x, y, w, h)

Classification + Localization: Do both

Idea #1: Localization as Regression

Input: image



Neural Net
→

Output:
Box coordinates
(4 numbers)

Correct output:
box coordinates
(4 numbers)

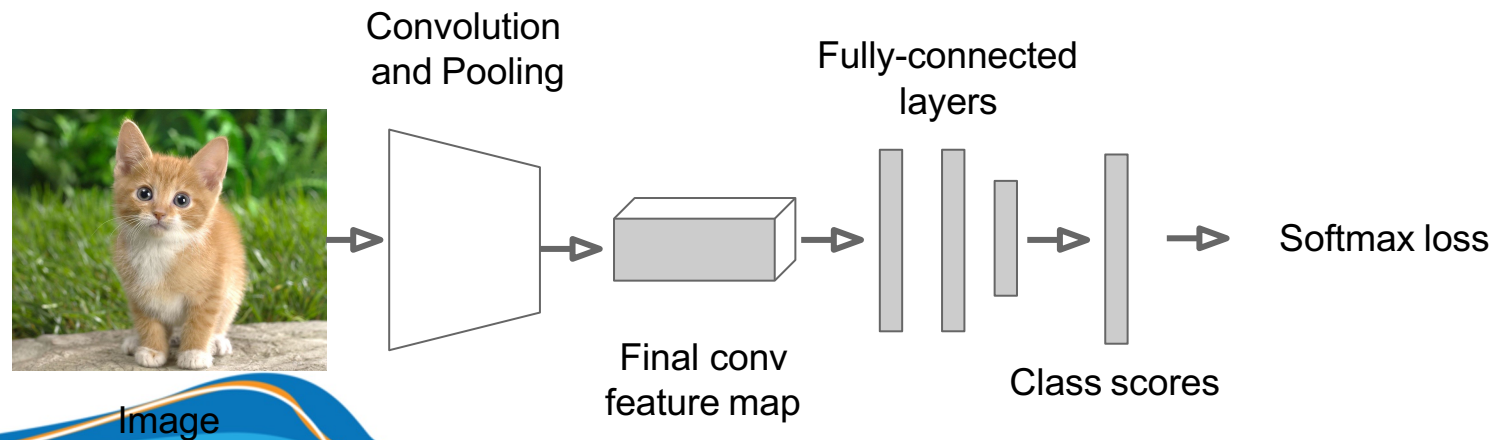
Loss:
L2 distance

Only one object,
simpler than detection



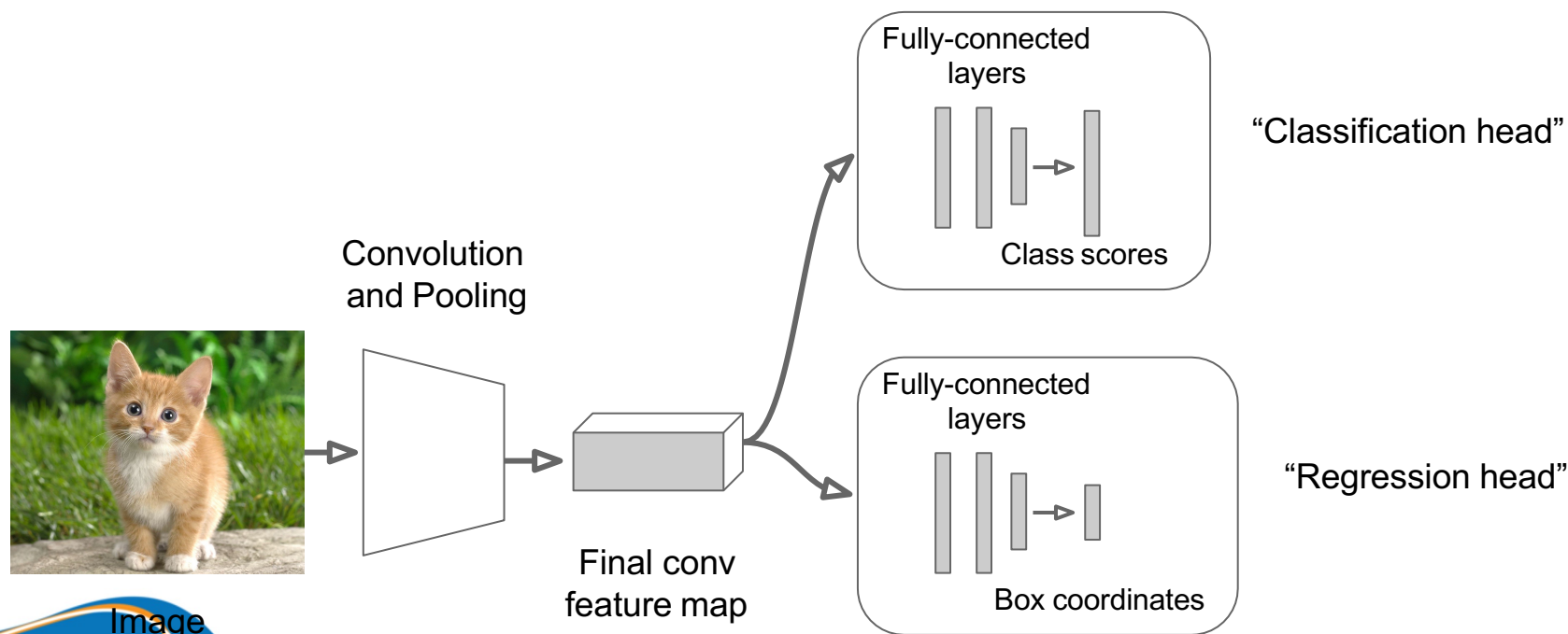
Simple Recipe for Classification + Localization

Step 1: Train (or download) a classification model (AlexNet, VGG, GoogLeNet)



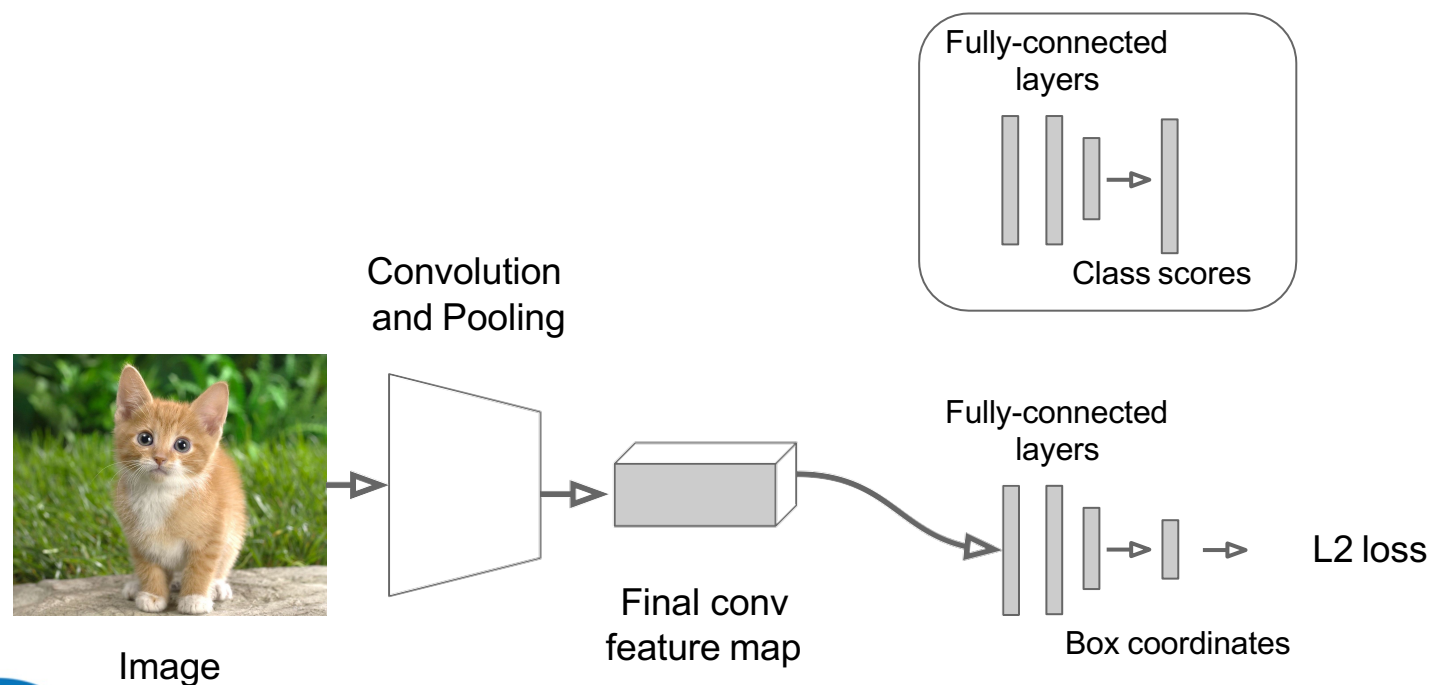
Simple Recipe for Classification + Localization

Step 2: Attach new fully-connected “regression head” to the network



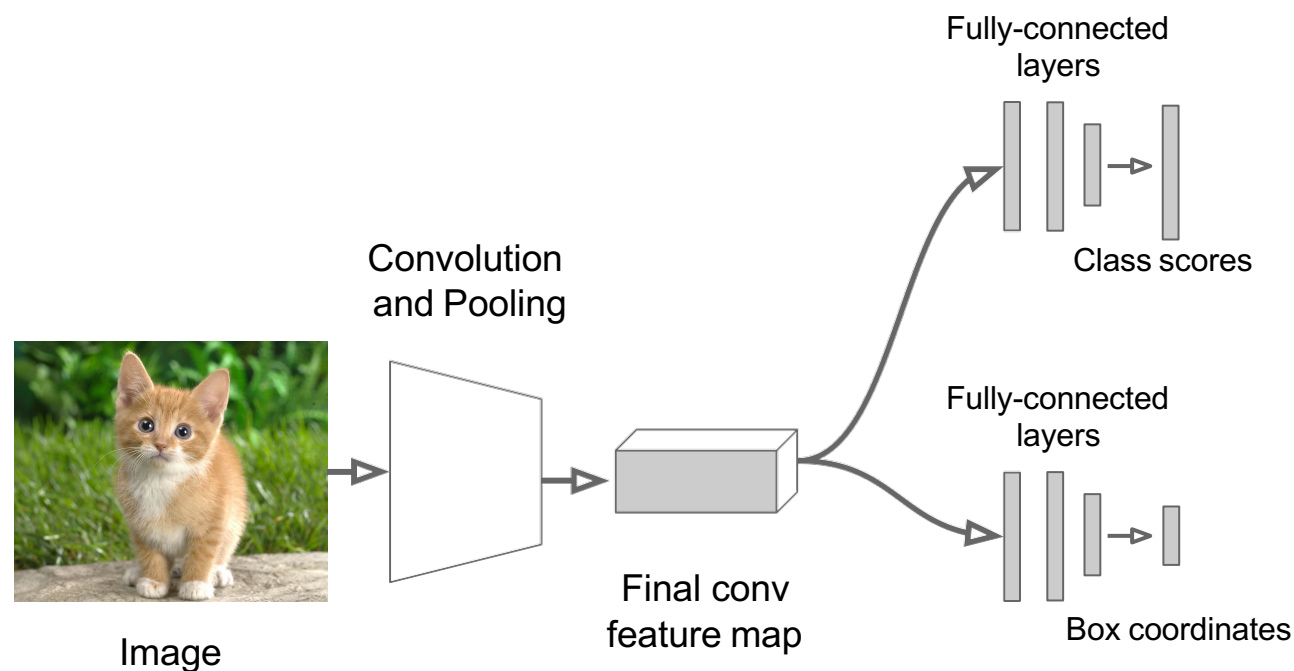
Simple Recipe for Classification + Localization

Step 3: Train the regression head only with SGD and L2 loss



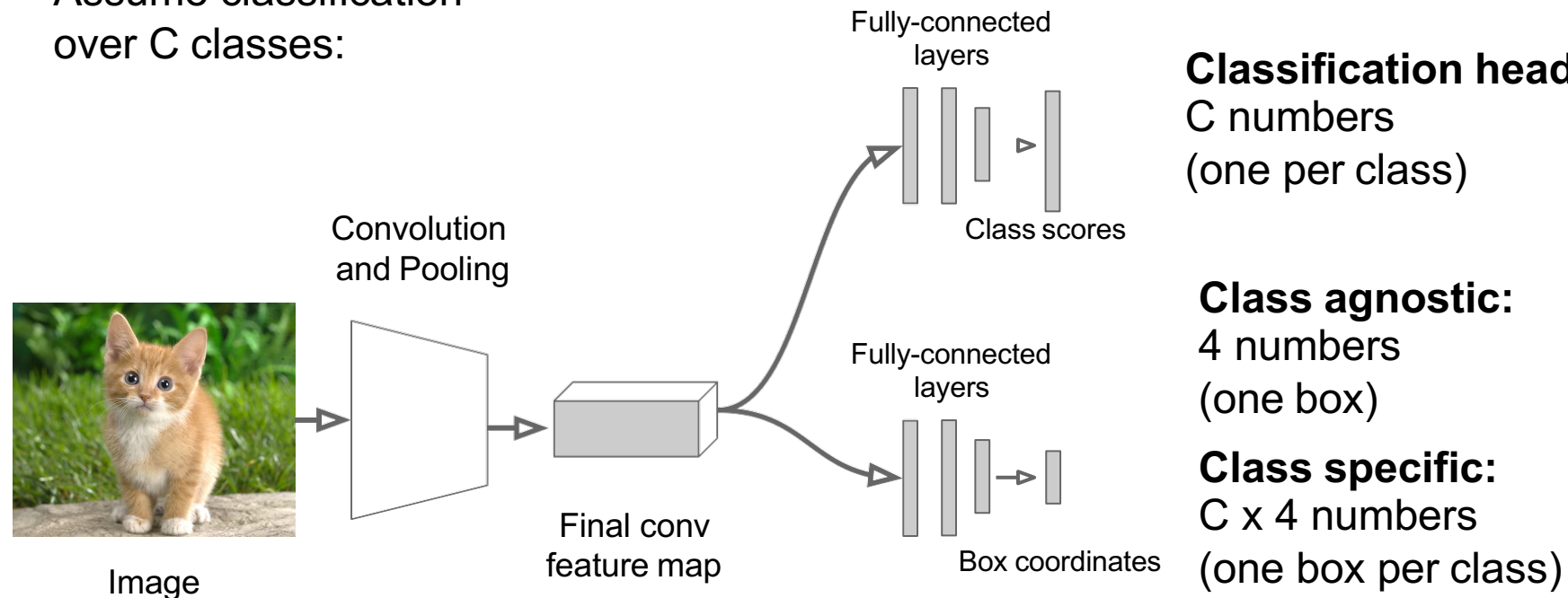
Simple Recipe for Classification + Localization

Step 4: At test time use both heads



Per-class vs class agnostic regression

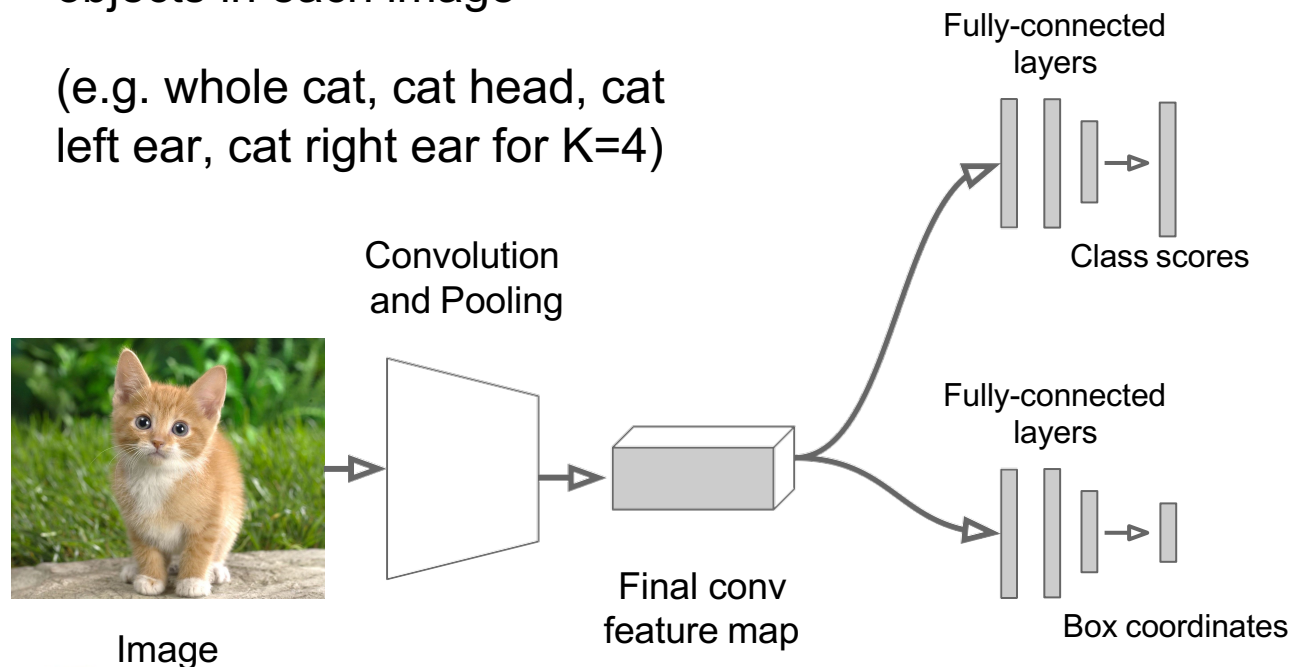
Assume classification
over C classes:



Aside: Localizing multiple objects

Want to localize **exactly** K objects in each image

(e.g. whole cat, cat head, cat left ear, cat right ear for $K=4$)



$K \times 4$ numbers
(one box per object)

Idea #2: Sliding Window

- Run classification + regression network at multiple locations on a high resolution image
- Convert fully-connected layers into convolutional layers for efficient computation
- Combine classifier and regressor predictions across all scales for final prediction



Sliding Window

Winner of ILSVRC 2013
localization challenge

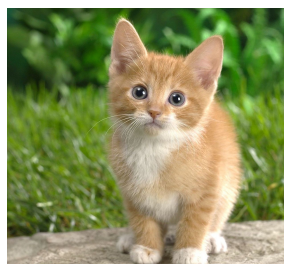
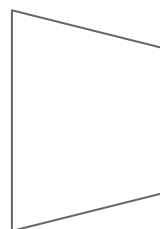
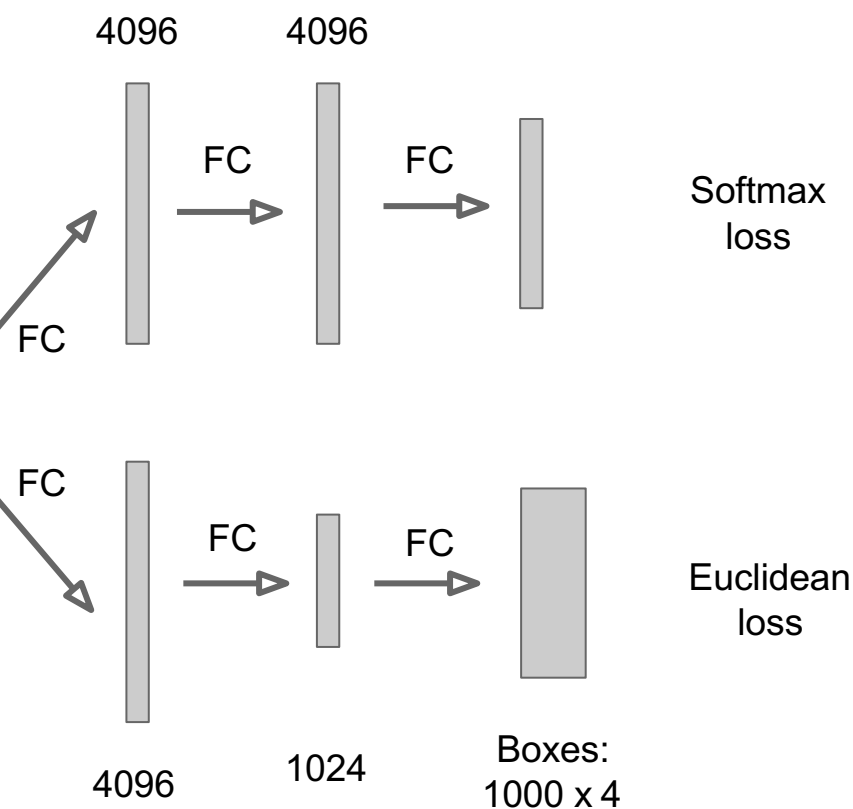


Image:
3 x 221 x 221

Convolution
+ pooling

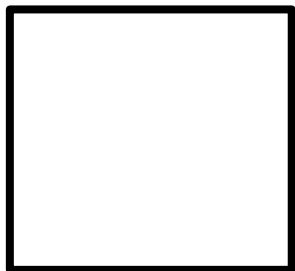


Feature map:
1024 x 5 x 5



Sermanet et al, "Integrated Recognition,
Localization and Detection using
Convolutional Networks", ICLR 2014

Sliding Window: Overfeat



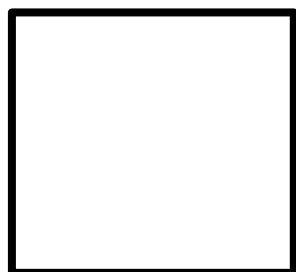
Network input:
3 x 221 x 221



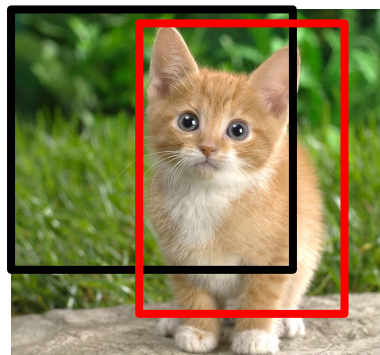
Larger image:
3 x 257 x 257



Sliding Window: Overfeat



Network input:
 $3 \times 221 \times 221$

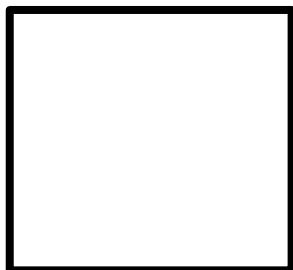


Larger image:
 $3 \times 257 \times 257$

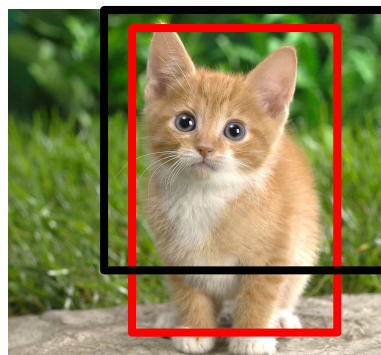
0.5	

Classification scores:
 $P(\text{cat})$

Sliding Window: Overfeat



Network input:
 $3 \times 221 \times 221$

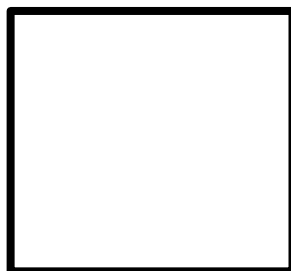


Larger image:
 $3 \times 257 \times 257$

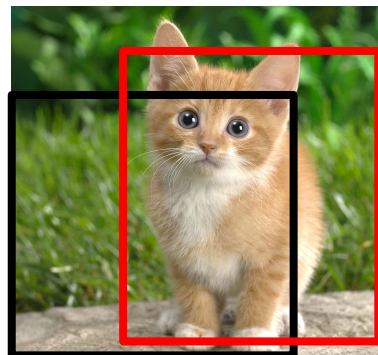
0.5	0.75

Classification scores:
 $P(\text{cat})$

Sliding Window: Overfeat



Network input:
 $3 \times 221 \times 221$

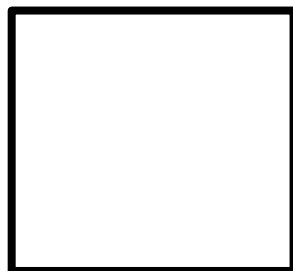


Larger image:
 $3 \times 257 \times 257$

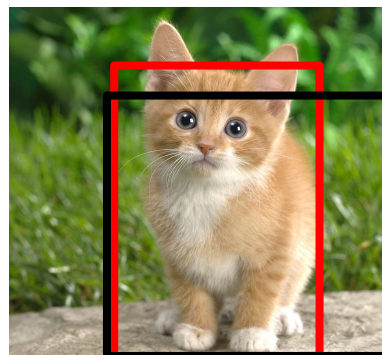
0.5	0.75
0.6	

Classification scores:
 $P(\text{cat})$

Sliding Window: Overfeat



Network input:
 $3 \times 221 \times 221$

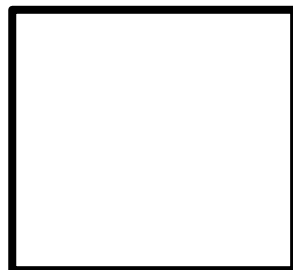


Larger image:
 $3 \times 257 \times 257$

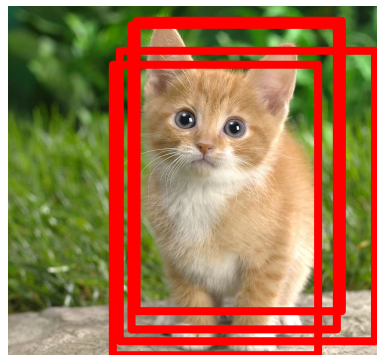
0.5	0.75
0.6	0.8

Classification scores:
 $P(\text{cat})$

Sliding Window: Overfeat



Network input:
 $3 \times 221 \times 221$



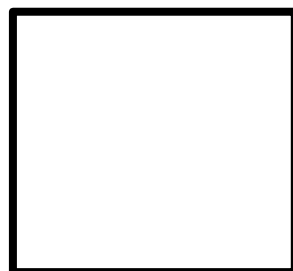
Larger image:
 $3 \times 257 \times 257$

0.5	0.75
0.6	0.8

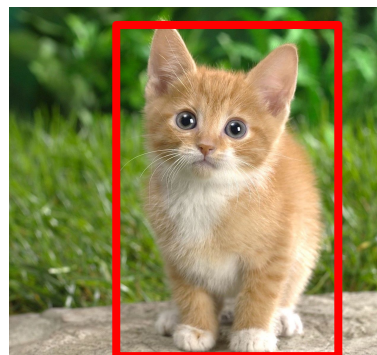
Classification scores:
 $P(\text{cat})$

Sliding Window: Overfeat

Greedy merge boxes and
scores (details in paper)



Network input:
3 x 221 x 221



Larger image:
3 x 257 x 257

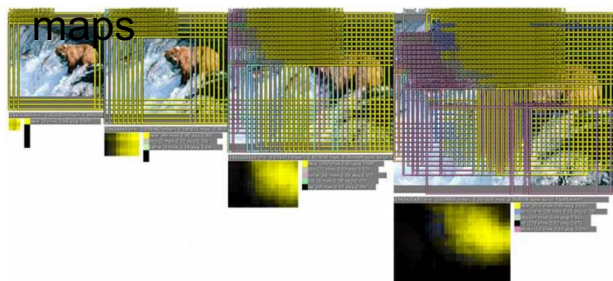
0.8

Classification score: P
(cat)

Sliding Window: Overfeat

In practice use many sliding window locations and multiple scales

Window positions + score maps



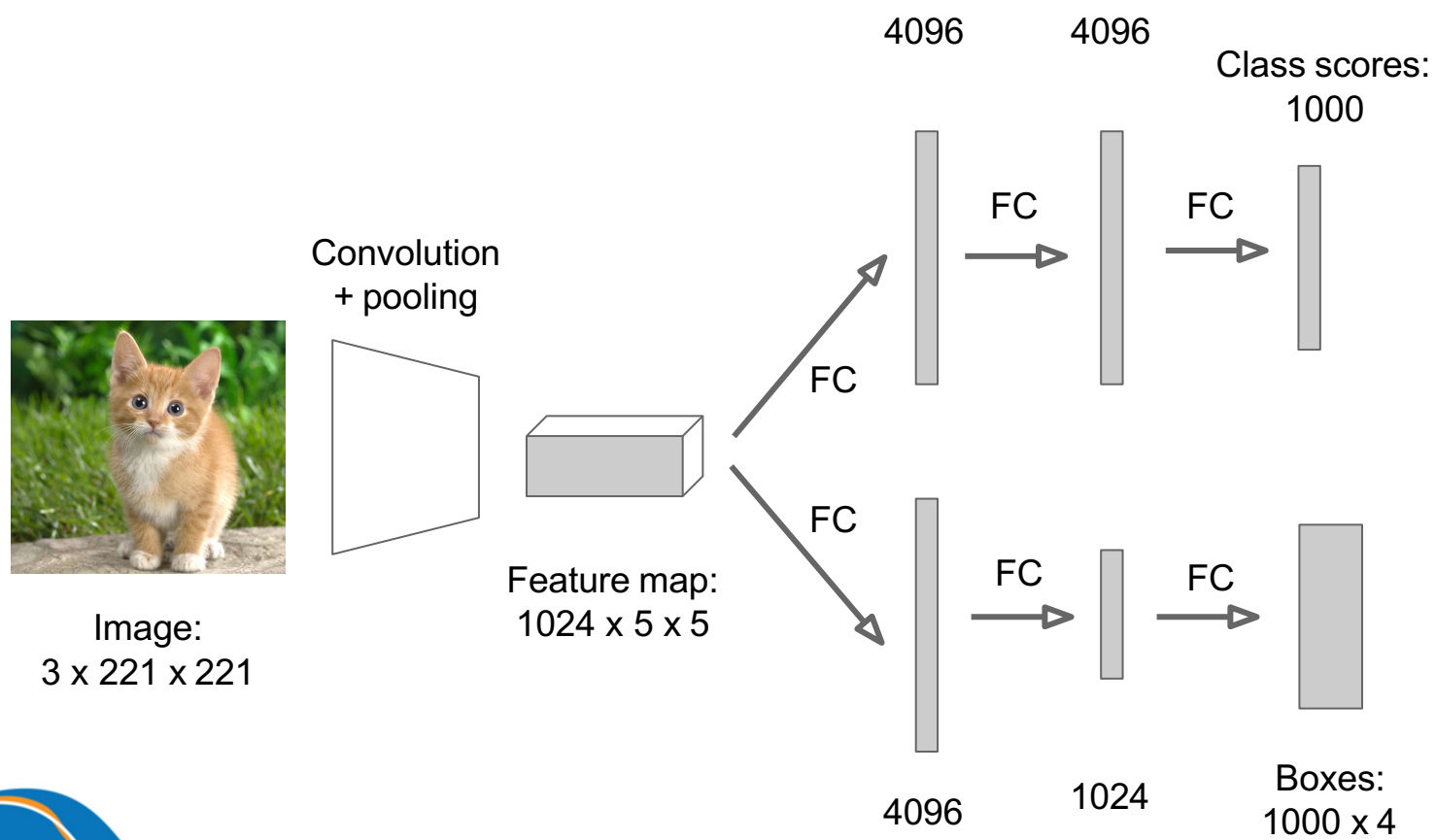
Box regression outputs



Final Predictions

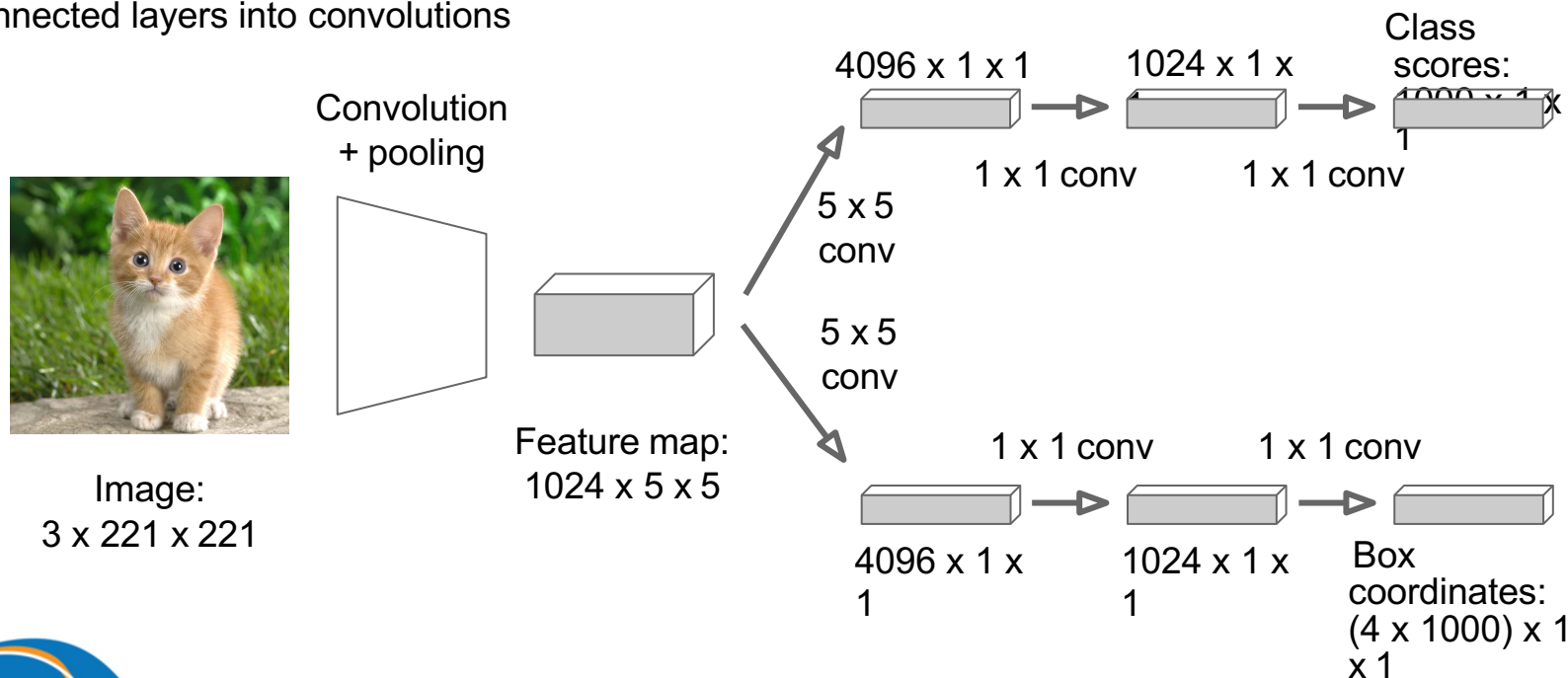


Efficient Sliding Window: Overfeat



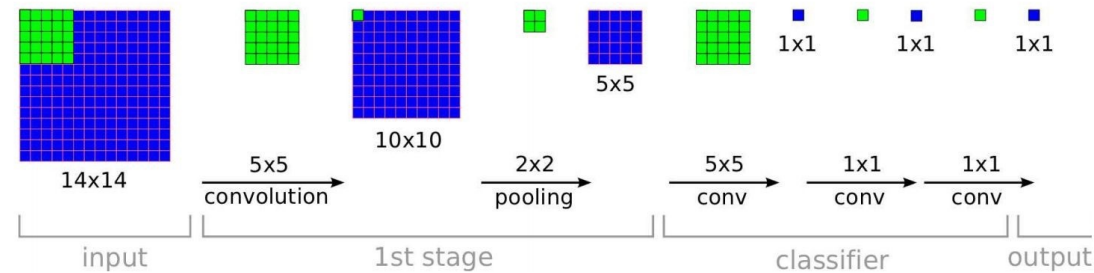
Efficient Sliding Window: Overfeat

Efficient sliding window by converting fully-connected layers into convolutions

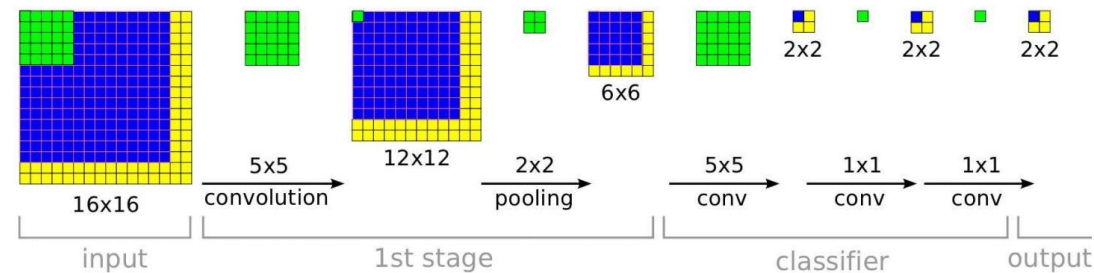


Efficient Sliding Window: Overfeat

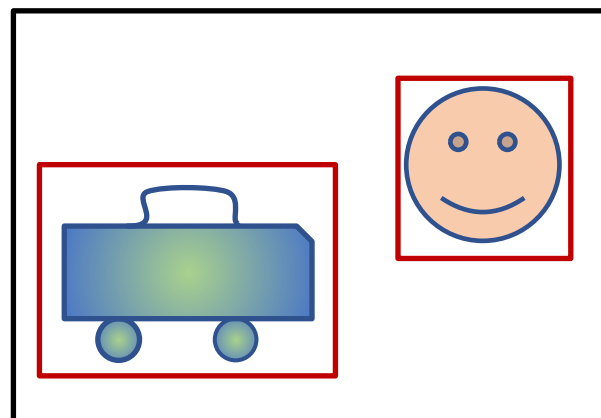
Training time: Small image, 1 x 1 classifier output



Test time: Larger image, 2 x 2 classifier output, only extra compute at yellow regions



Object Detection



Car, Person

Detection as Regression?



DOG, (x, y, w, h)
CAT, (x, y, w, h)
CAT, (x, y, w, h)
DUCK (x, y, w, h)

= 16 numbers

Detection as Regression?



DOG, (x, y, w, h)

CAT, (x, y, w, h)

= 8 numbers

Detection as Regression?



CAT, (x, y, w, h)

CAT, (x, y, w, h)

....

CAT (x, y, w, h)

= many numbers

Need variable sized outputs

Detection as Classification



CAT? NO

DOG? NO

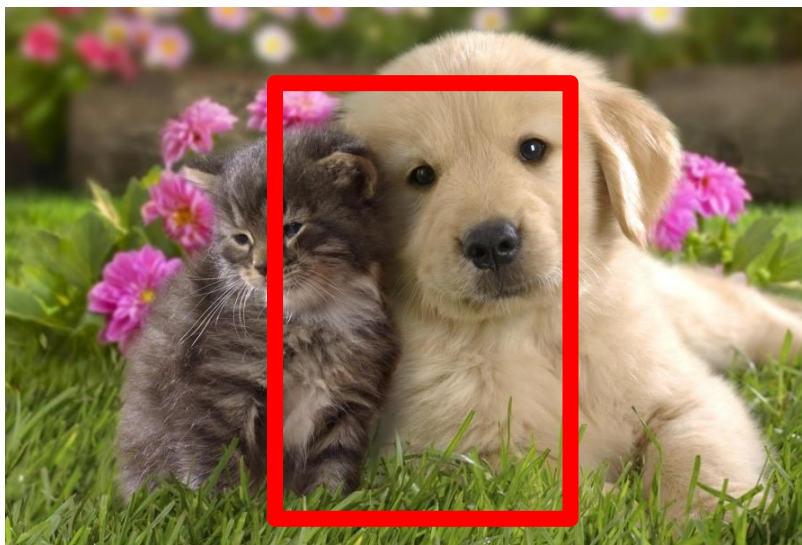
Detection as Classification



CAT? YES!

DOG? NO

Detection as Classification



CAT? NO

DOG? NO

Detection as Classification

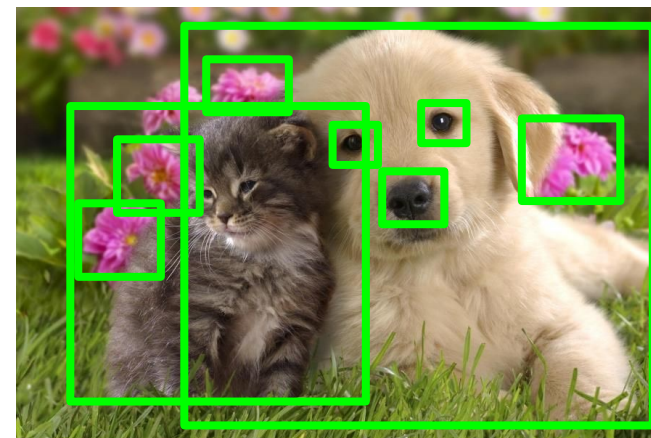
Problem: Need to test many positions and scales, and use a computationally demanding classifier (CNN)

Solution: Only look at a tiny subset of possible positions



Region Proposals

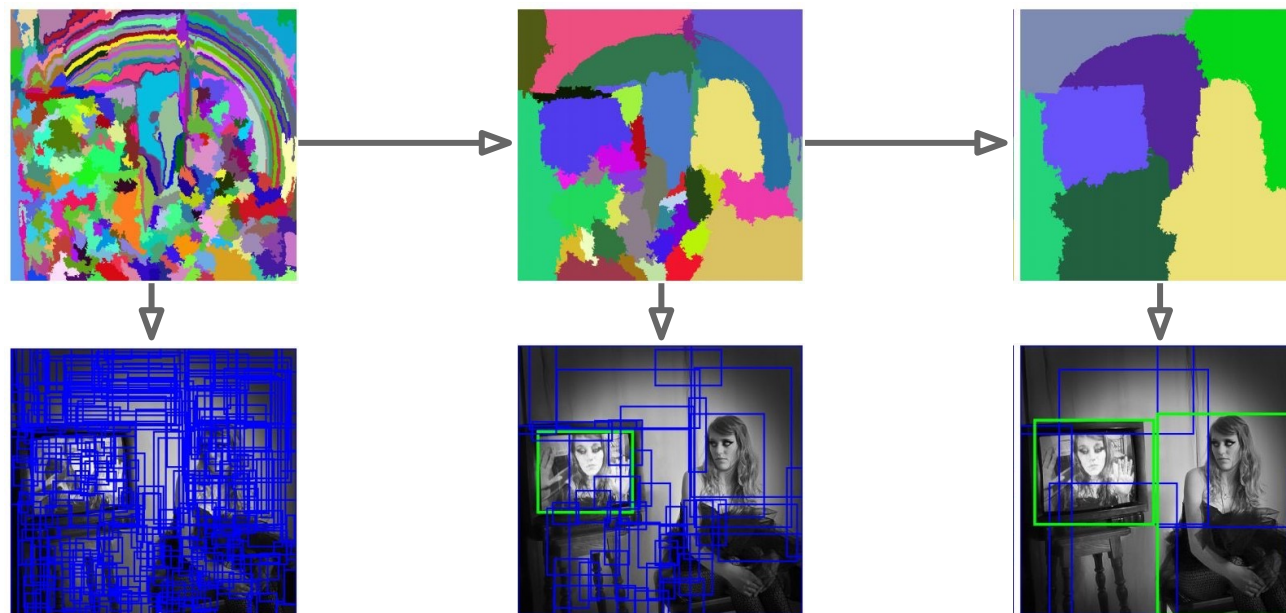
- Find “blobby” image regions that are likely to contain objects
- “Class-agnostic” object detector
- Look for “blob-like” regions



Region Proposals: Selective Search

Bottom-up segmentation, merging regions at multiple scales

Convert
regions
to boxes



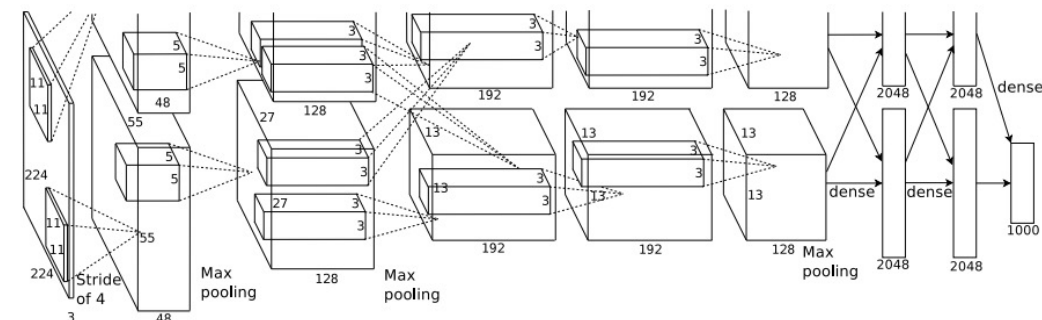
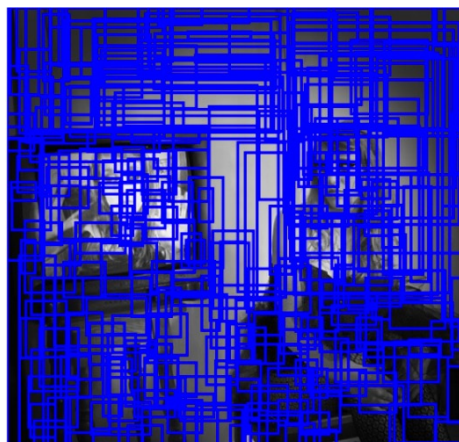
Region-based Convolutional Network (R-CNN)

Topics In Computer Vision



R-CNN

R-CNN = Region Proposal + CNN



R-CNN



Input image

R-CNN

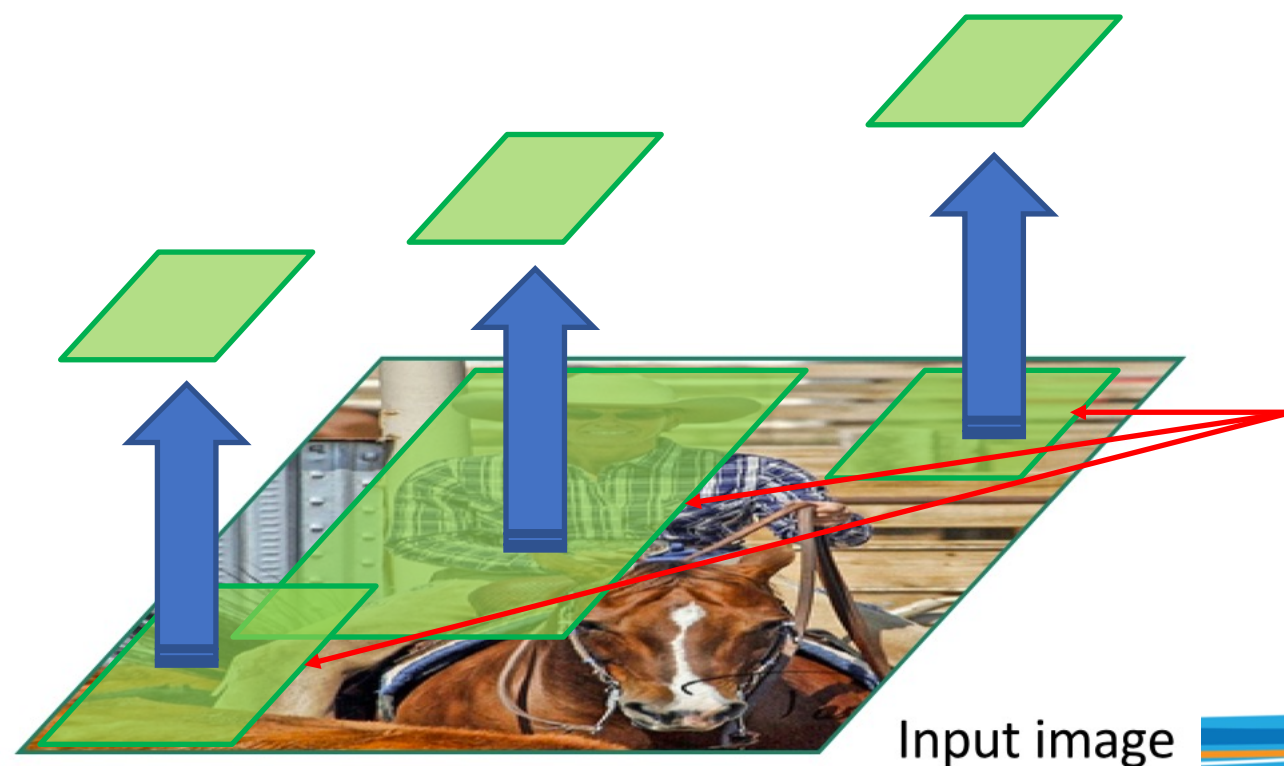
Warped Image Region

Region of Interest
(ROI) from a
proposal method
(~2K)



Input image

R-CNN

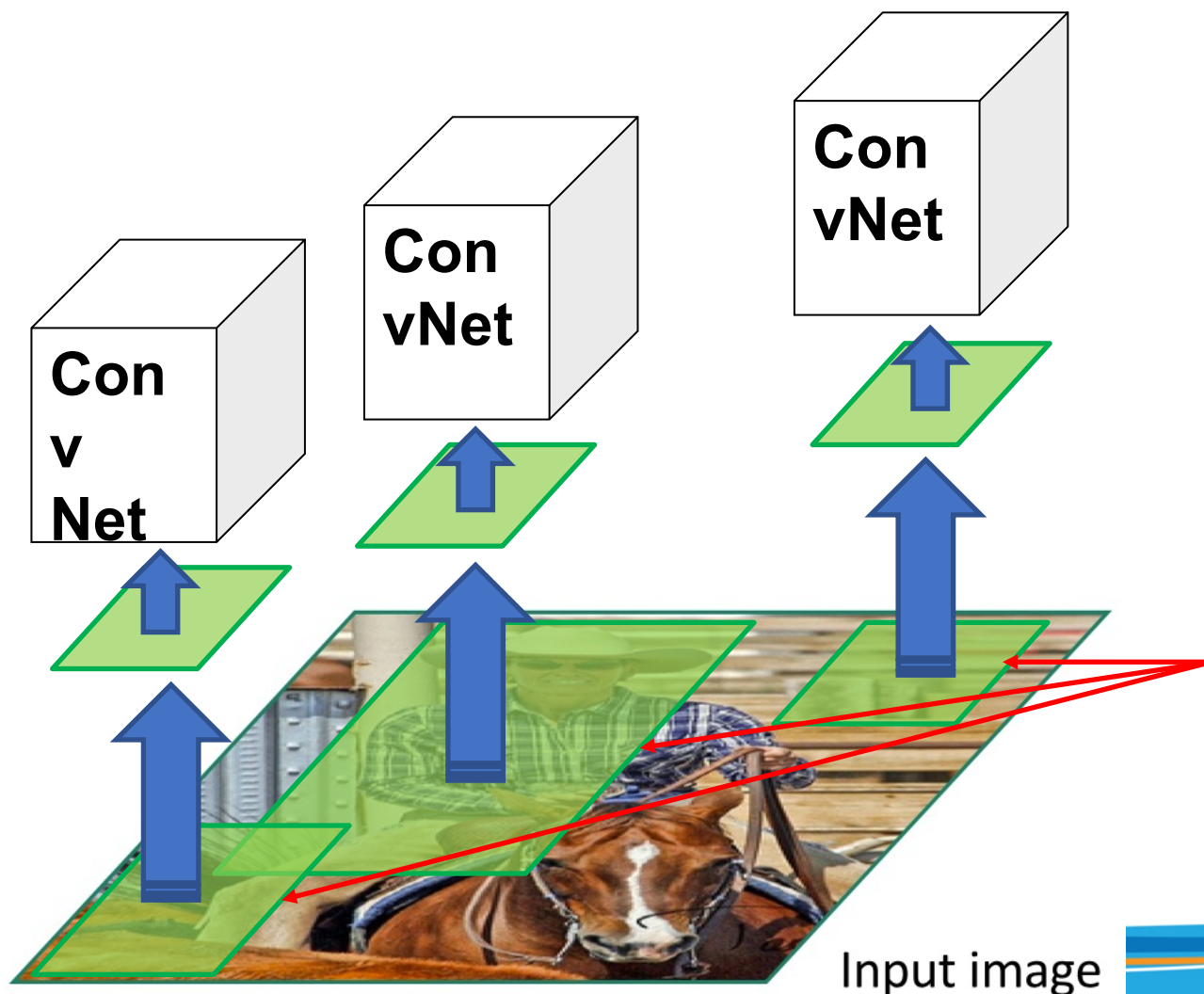


Warped Image Region

Region of Interest
(ROI) from a
proposal method
(~2K)

Input image

R-CNN

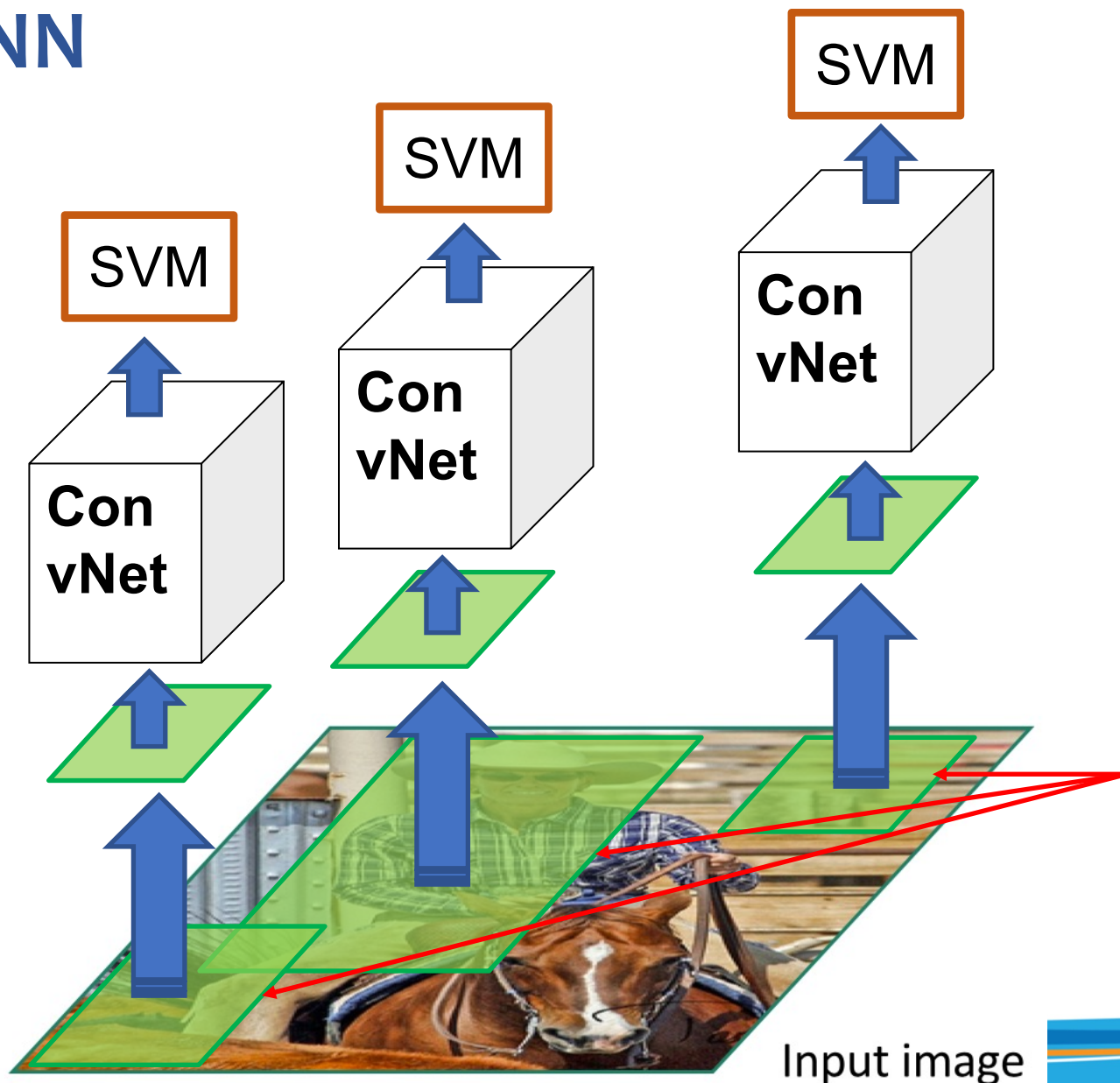


Forward each region
through ConvNet

Warped Image Region

Region of Interest
(ROI) from a
proposal method
(~2K)

R-CNN



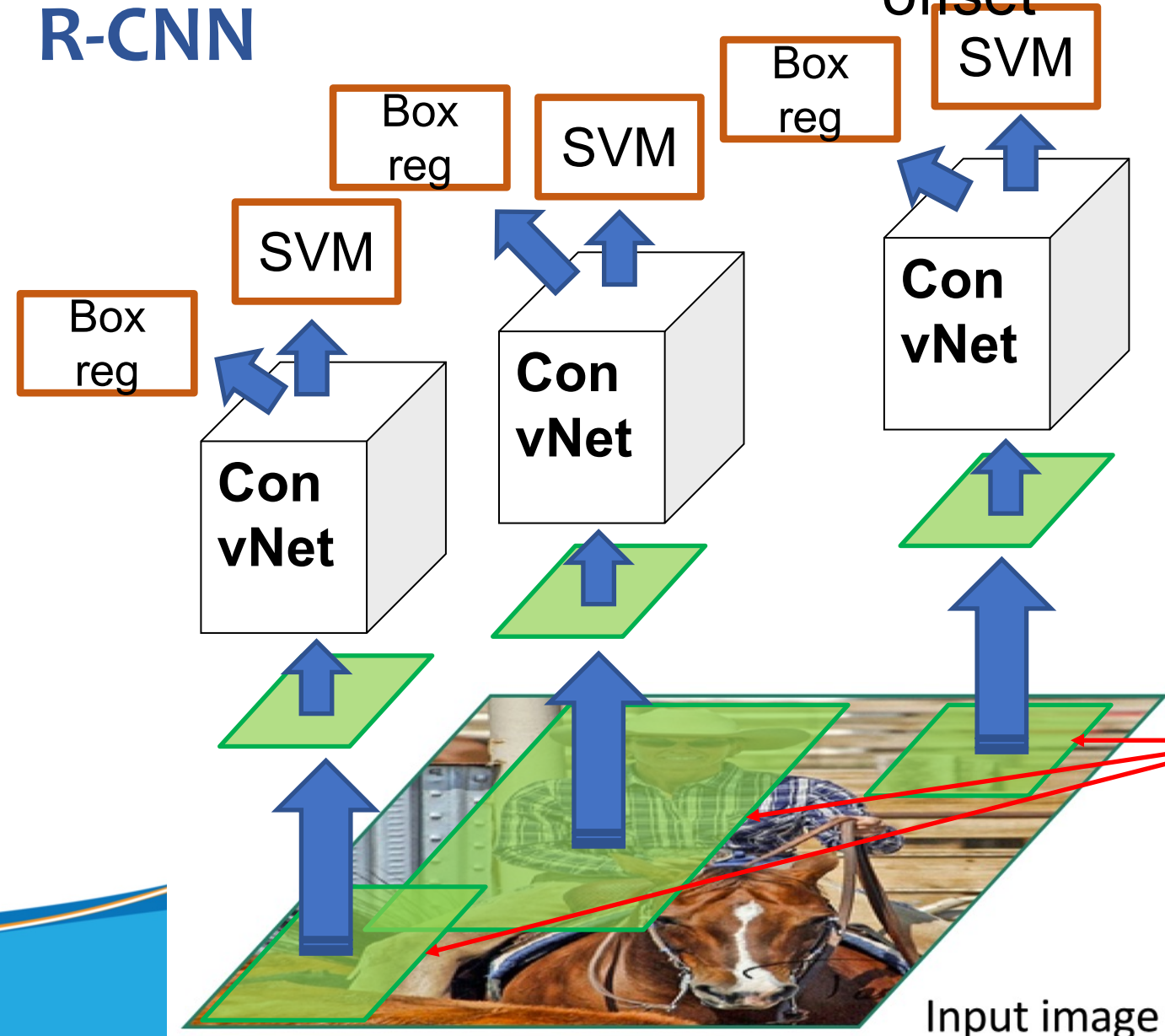
Classify regions with SVMs

Forward each region through ConvNet

Warped Image Region

Region of Interest (ROI) from a proposal method (~2K)

R-CNN



Classify regions with SVMs

Forward each region through ConvNet

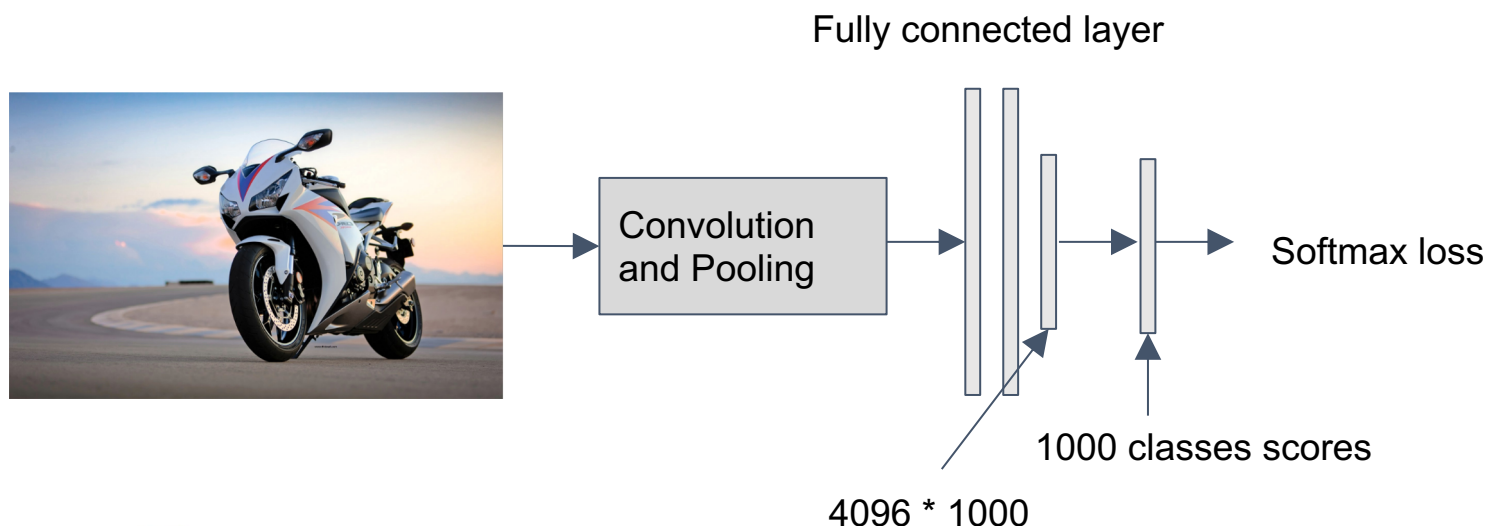
Warped Image Region

Region of Interest (ROI) from a proposal method (~2K)

Linear regression for bounding box offset

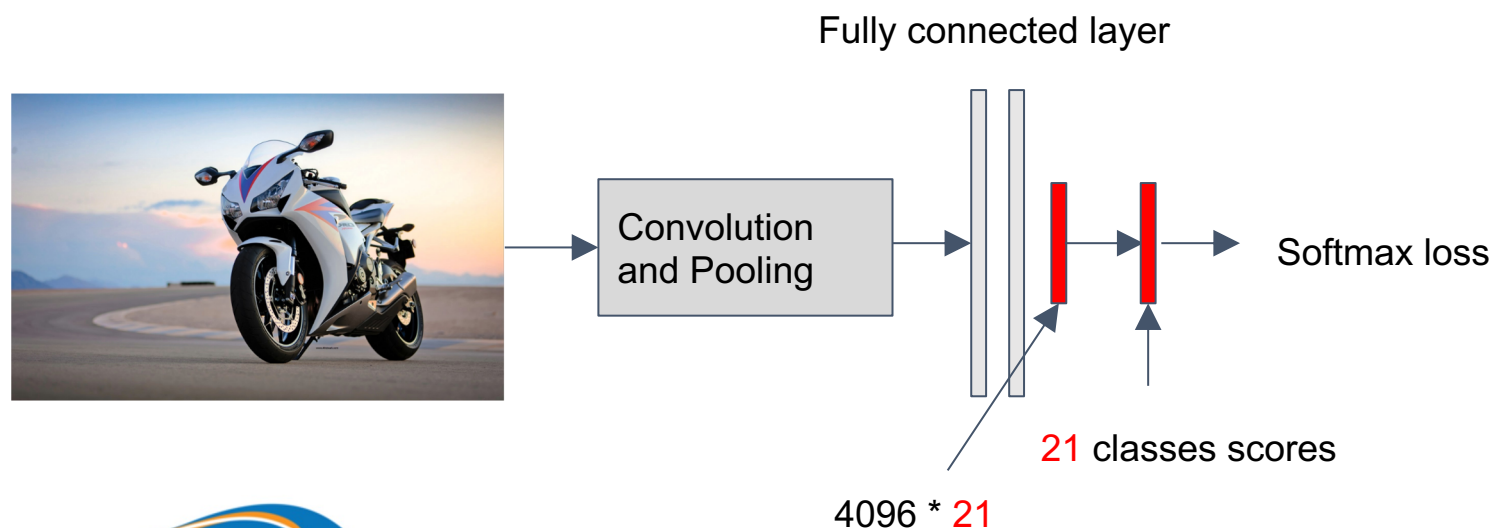
Training RCNN

- Step1: train your own CNN model for classification (or use existing model), using ImageNet dataset.



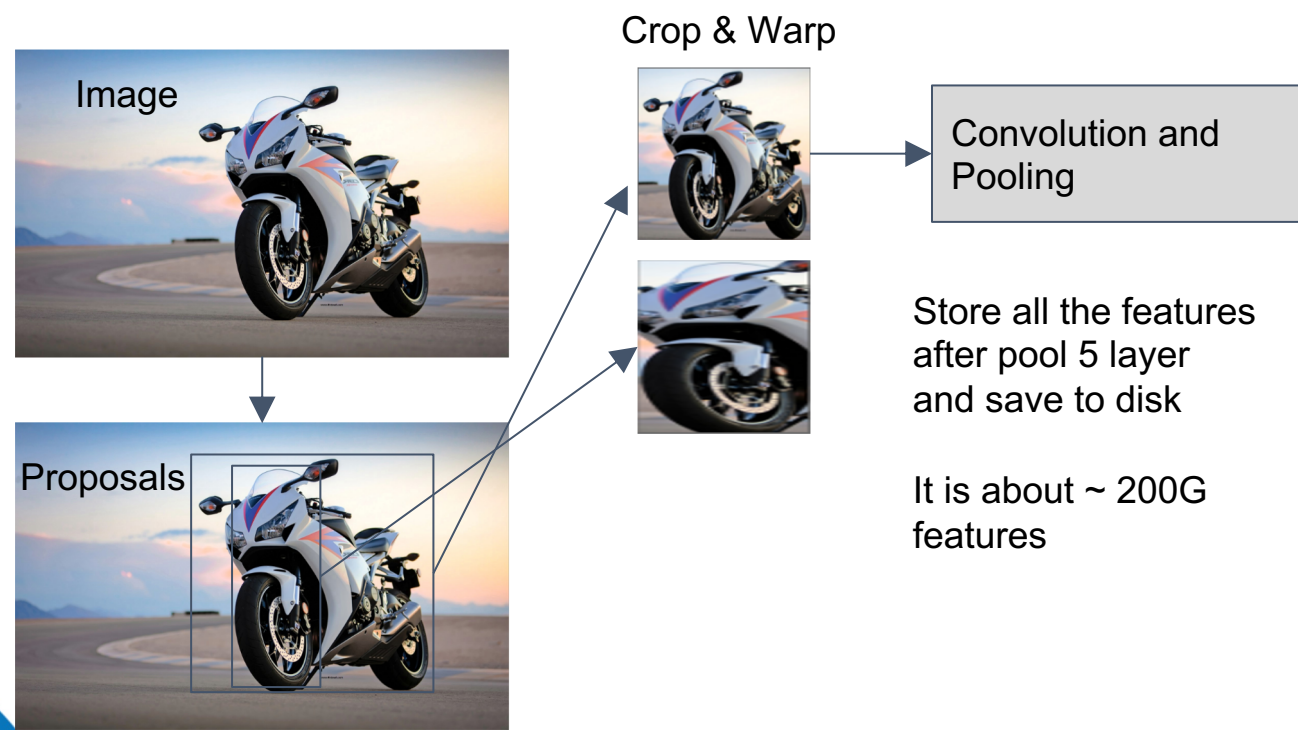
Training RCNN

- Step2: focus on 20 classes + 1 background. Remove the last FC layer and replace it with a smaller layer and fine-tune the model using PASCAL VOC dataset



Training RCNN

- Step3: extract feature



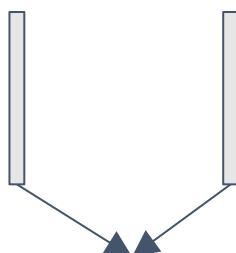
Training RCNN

- Step4: train SVM for each class

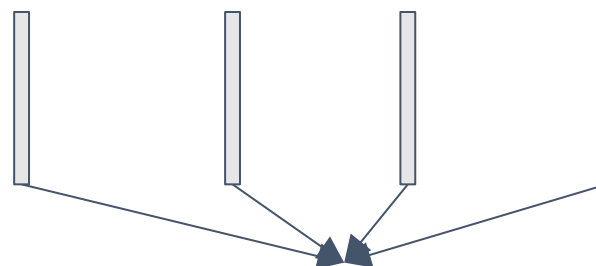
Crop /
Warp
image



Features
from last
step



Positive
samples for
Motorbike



Negative
samples for
Motorbike

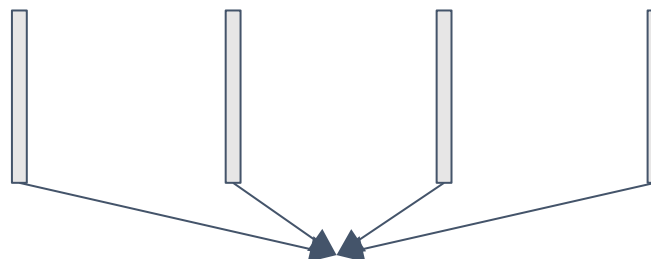
Training RCNN

- Step4: train SVM for each class

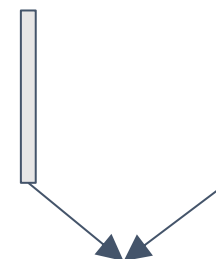
Crop /
Warp
image



Features
from last
step



Negative
samples for
Bicycle



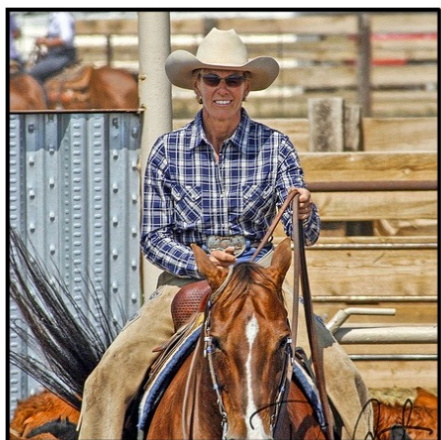
Positive
samples for
Bicycle

Sumanry: RCNN

1. Generate category independent region proposals
2. Extract a fixed length feature vectors from CNN
3. Class specific linear SVM



Summary: R-CNN

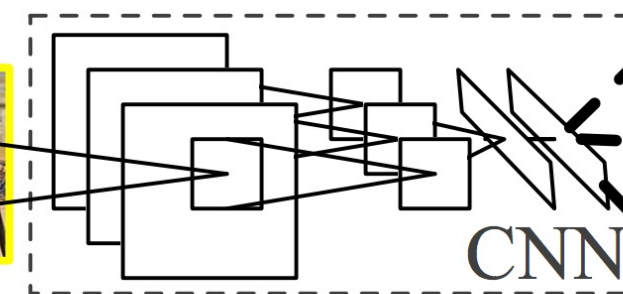


1. Input image



2. Extract region proposals (~2k)

warped region



3. Compute CNN features

aeroplane? no.

⋮

person? yes.

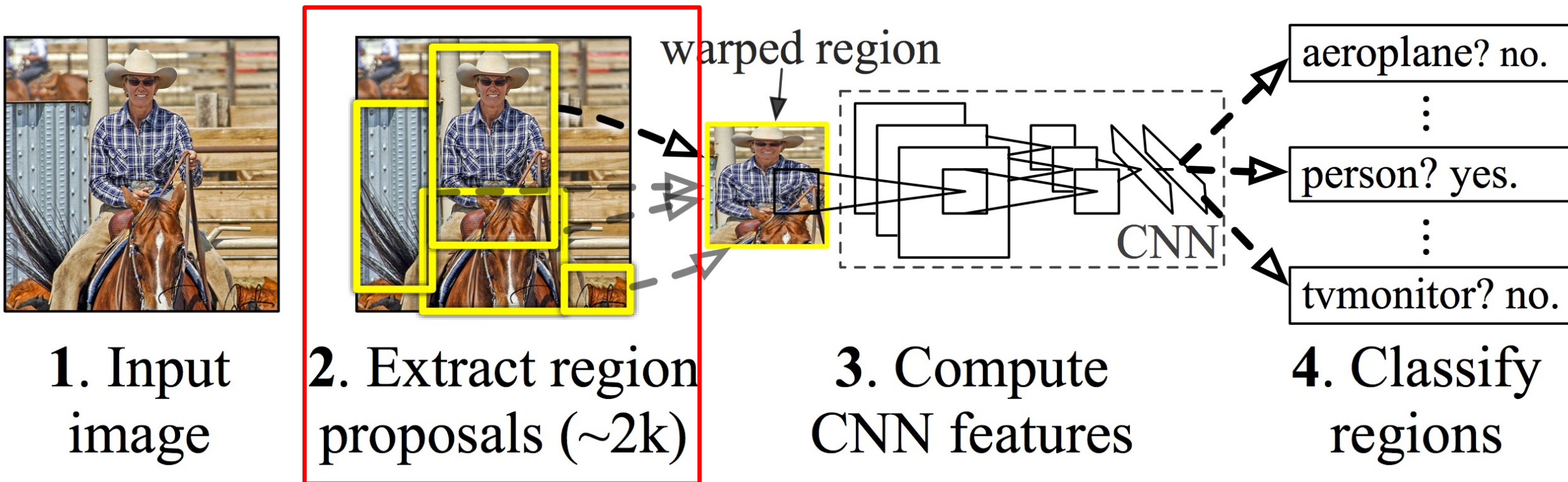
⋮

tvmonitor? no.

4. Classify regions

1. Generate category independent region proposals

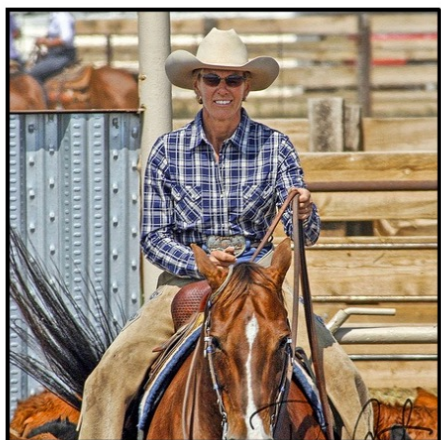
Summary: R-CNN



Many different algorithm exist

R-CNN uses superpixel based selective search algorithm

Summary: R-CNN

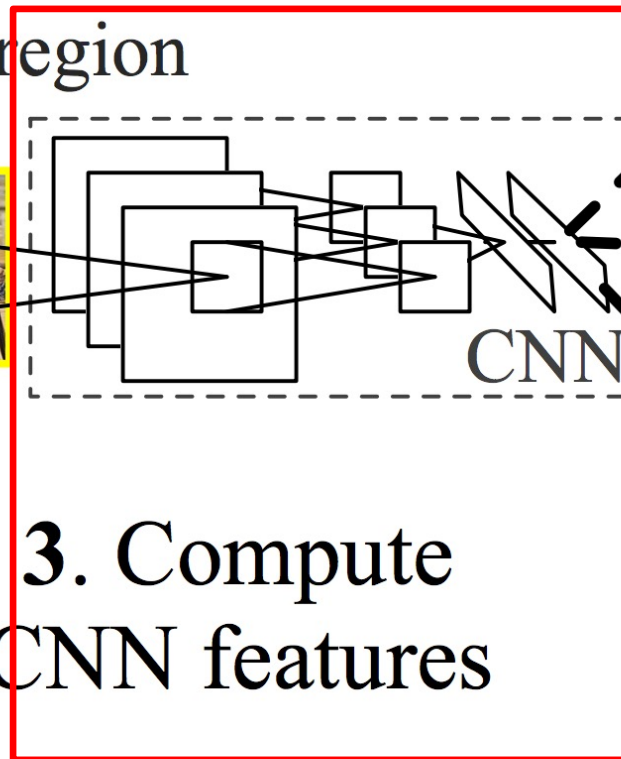


1. Input image



2. Extract region proposals (~2k)

warped region



3. Compute CNN features

aeroplane? no.

⋮

person? yes.

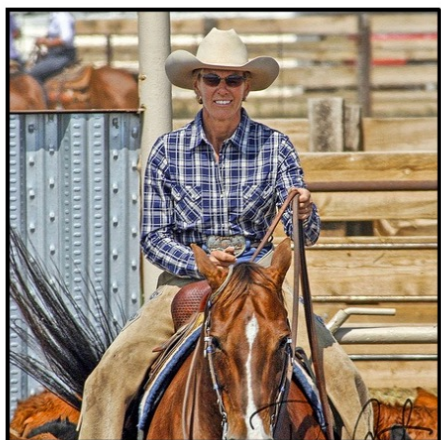
⋮

tvmonitor? no.

4. Classify regions

2. AlexNet is used to extract feature
4096 features are extracted

Summary: R-CNN

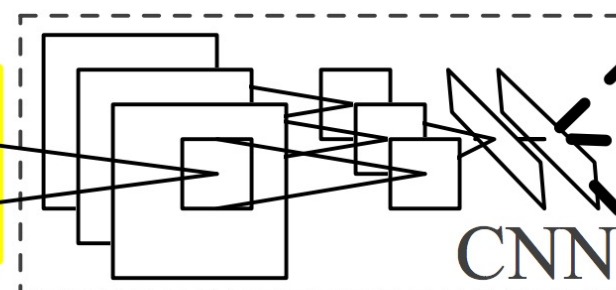


1. Input image

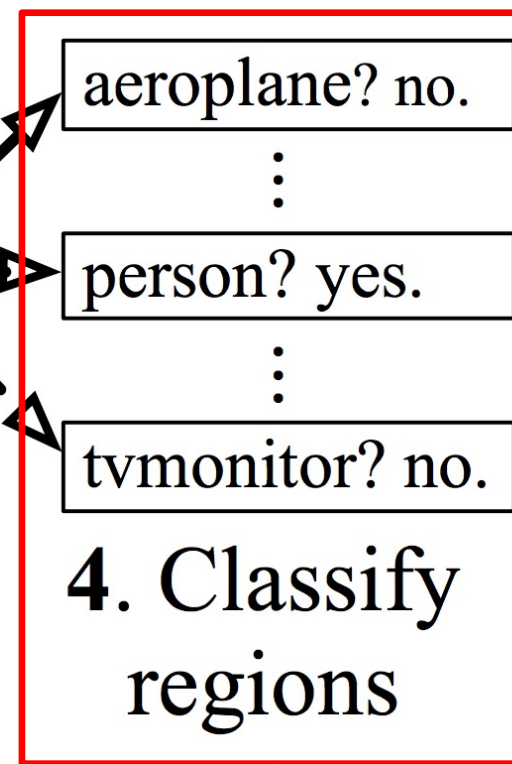


2. Extract region proposals (~2k)

warped region



3. Compute CNN features



3. Linear SVM

Questions and Answers